LAPORAN PRAKTIKUM TEKNOLOGI IOT II

TUGAS MODUL 6

PROTOKOL KOMUNIKASI TELEMETRI INDUSTRI(MQTT)

Disusun Oleh:

Nama : Naela Deswita

NPM : 25782106

Kelas : 3D-TRI

Dosen Pengampu:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

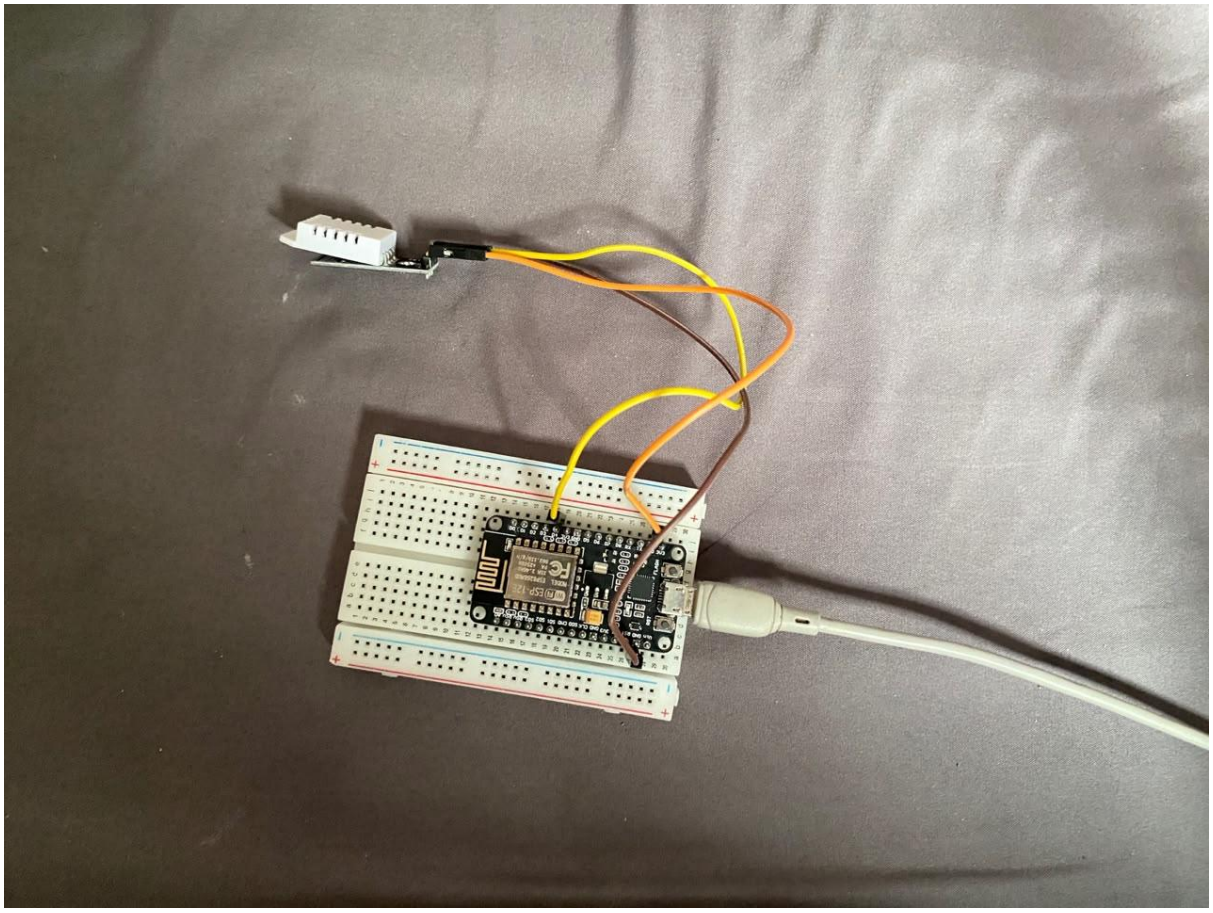
2026/2027

6.5 Praktikum

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

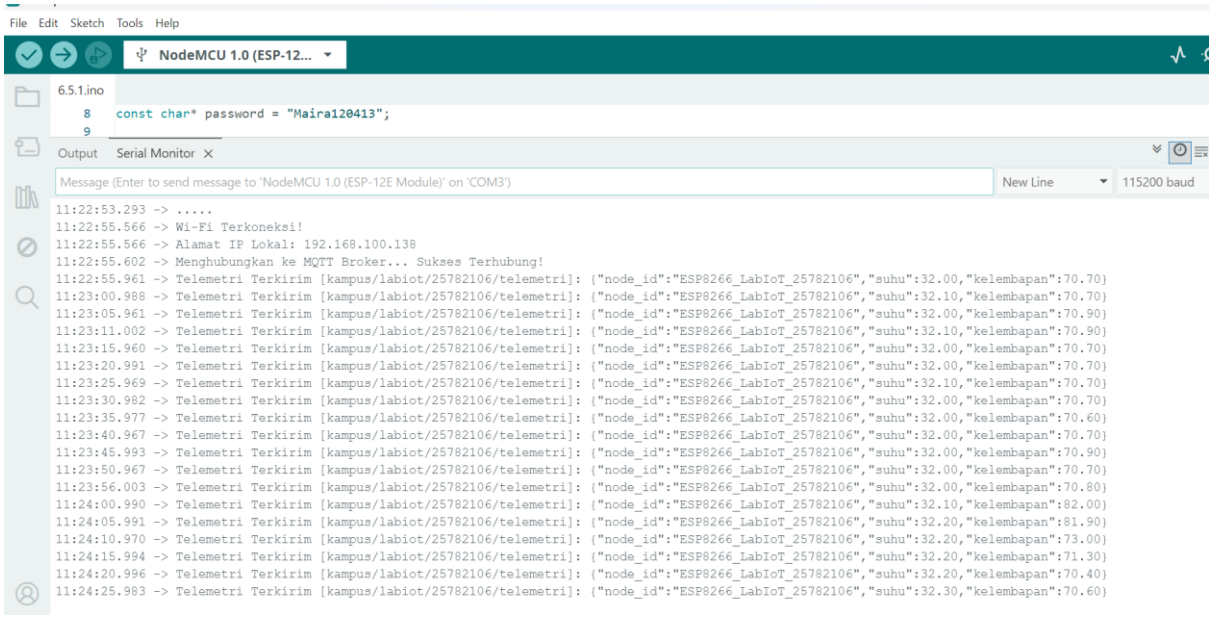
1. Rangkaian

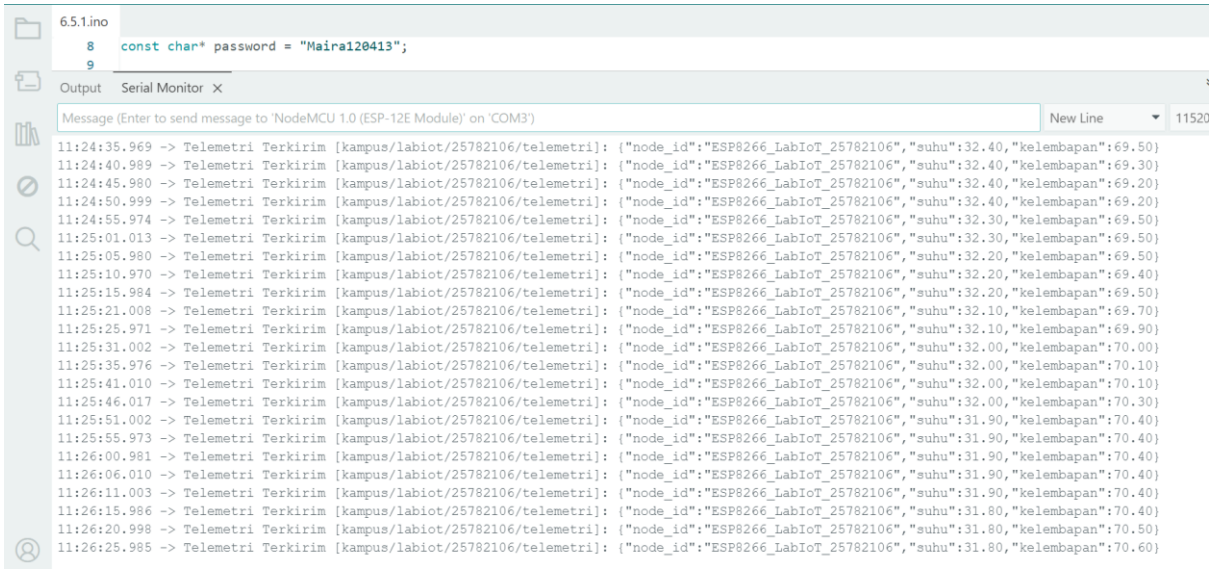
Rangkaian ini menggunakan NodeMCU ESP8266 yang dihubungkan dengan sensor DHT untuk membaca suhu dan kelembapan. ESP8266 terhubung ke laptop melalui kabel USB untuk menjalankan program dan melihat hasilnya.



2. Hasil Serial Monitor

ESP8266 berhasil terhubung ke Wi-Fi dan broker MQTT. Data suhu dan kelembapan berhasil dibaca dari sensor, kemudian dikirim dalam format JSON ke topic telemetri secara berkala. Hasil ini menunjukkan bahwa proses pengiriman data sensor melalui MQTT berhasil dilakukan.





3. Kode Program

Kode program digunakan untuk menghubungkan ESP8266 ke Wi-Fi dan broker MQTT, membaca sensor DHT, serta mengemas hasil pembacaan ke dalam format JSON sebelum dikirim ke broker.

6.5.1.ino

```ino
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi (Gunakan Hotspot Pribadi Anda)
const char* ssid = "Kosan Vama";
const char* password = "Maira120413";

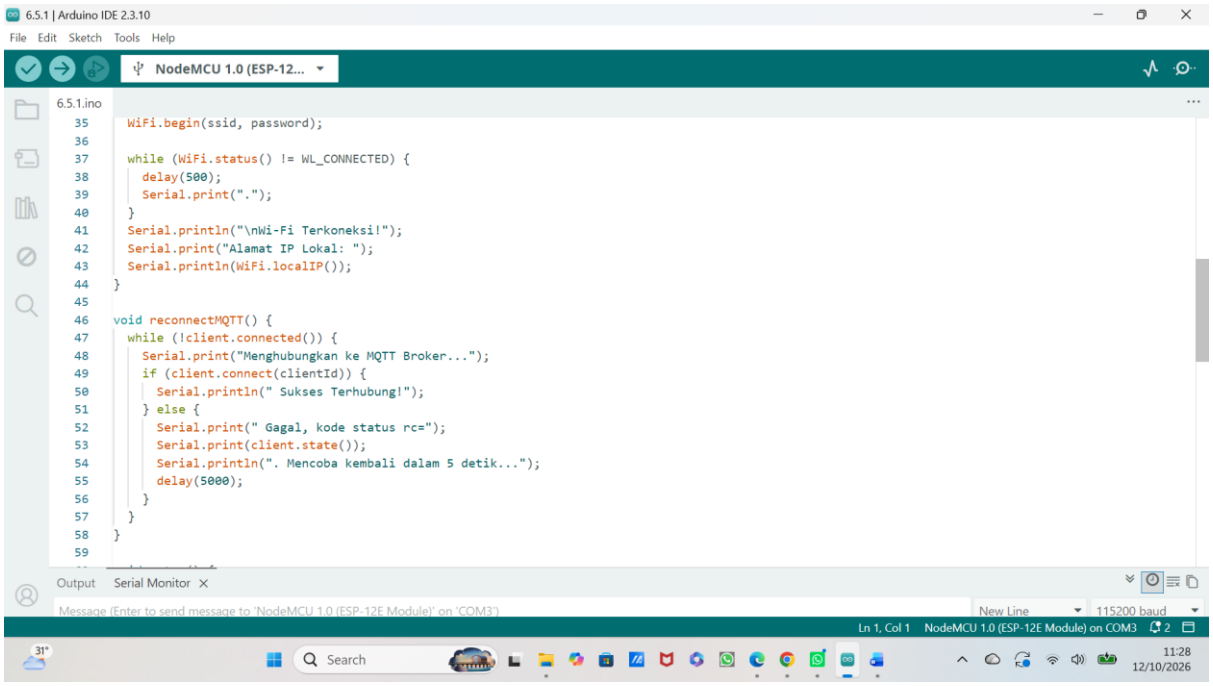
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

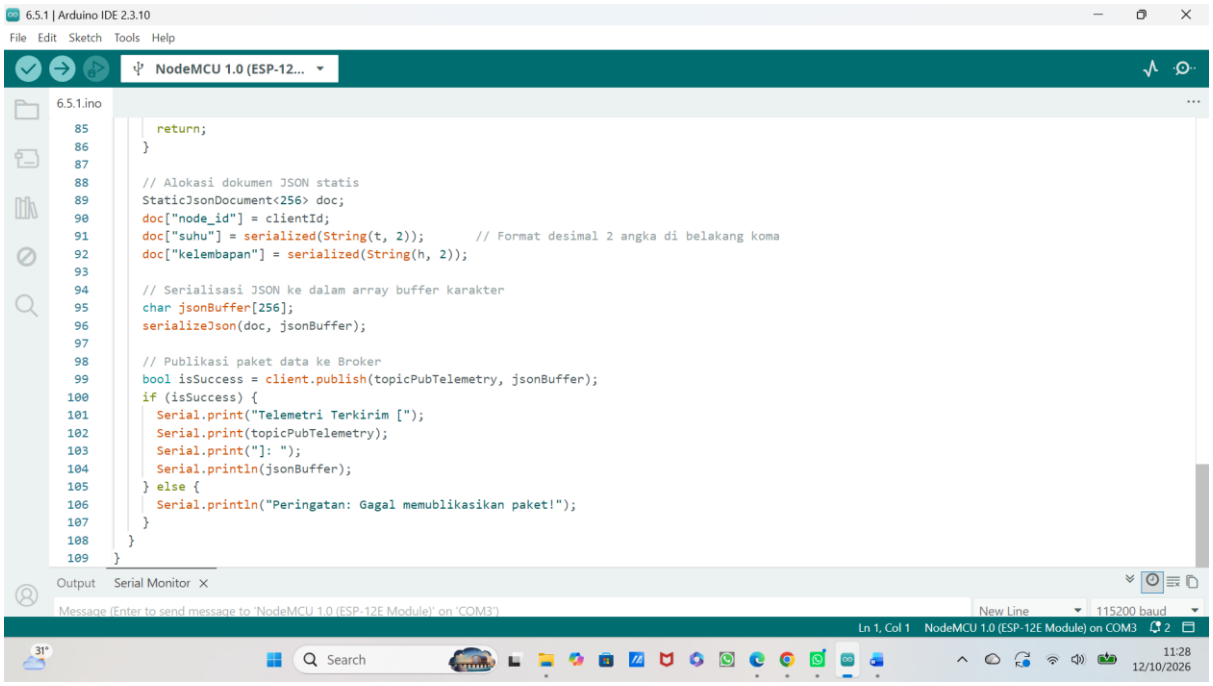
// IDENTITAS KLIEN (Wajib Unik! Ganti dengan NIM Anda agar tidak bertabrakan)
const char* clientId = "ESP8266_LabIoT_25782106";

// Definisi Topik Publikasi Telemetri Mandiri
const char* topicPubTelemetry = "kampus/labiot/25782106/telemetri";

// Konfigurasi Pin Sensor DHT
const byte dhtPin = 2;    // Pin D4 (GPIO 2)
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
```



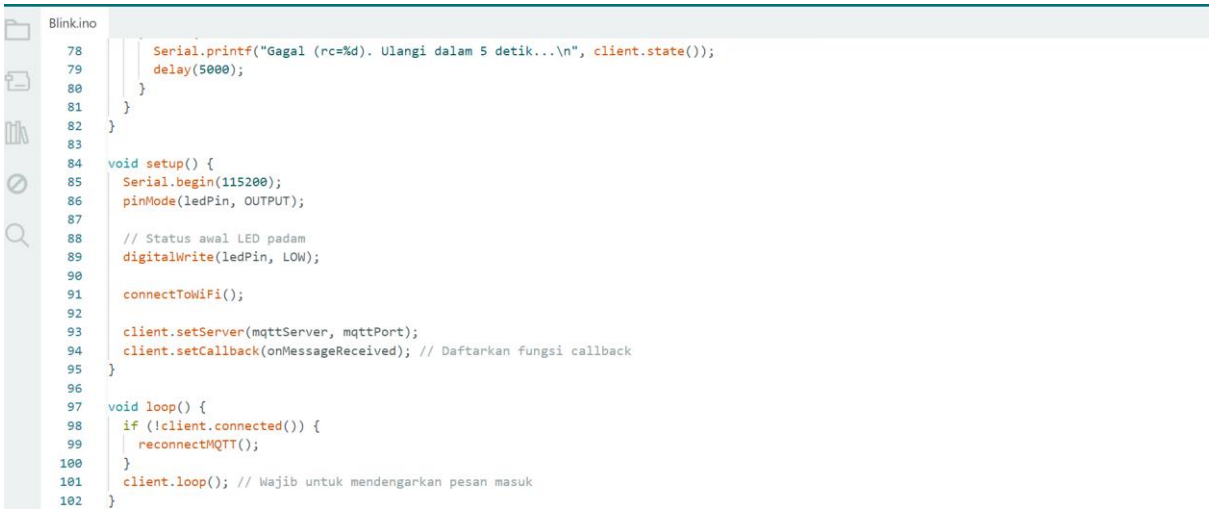


6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

1. Kode program

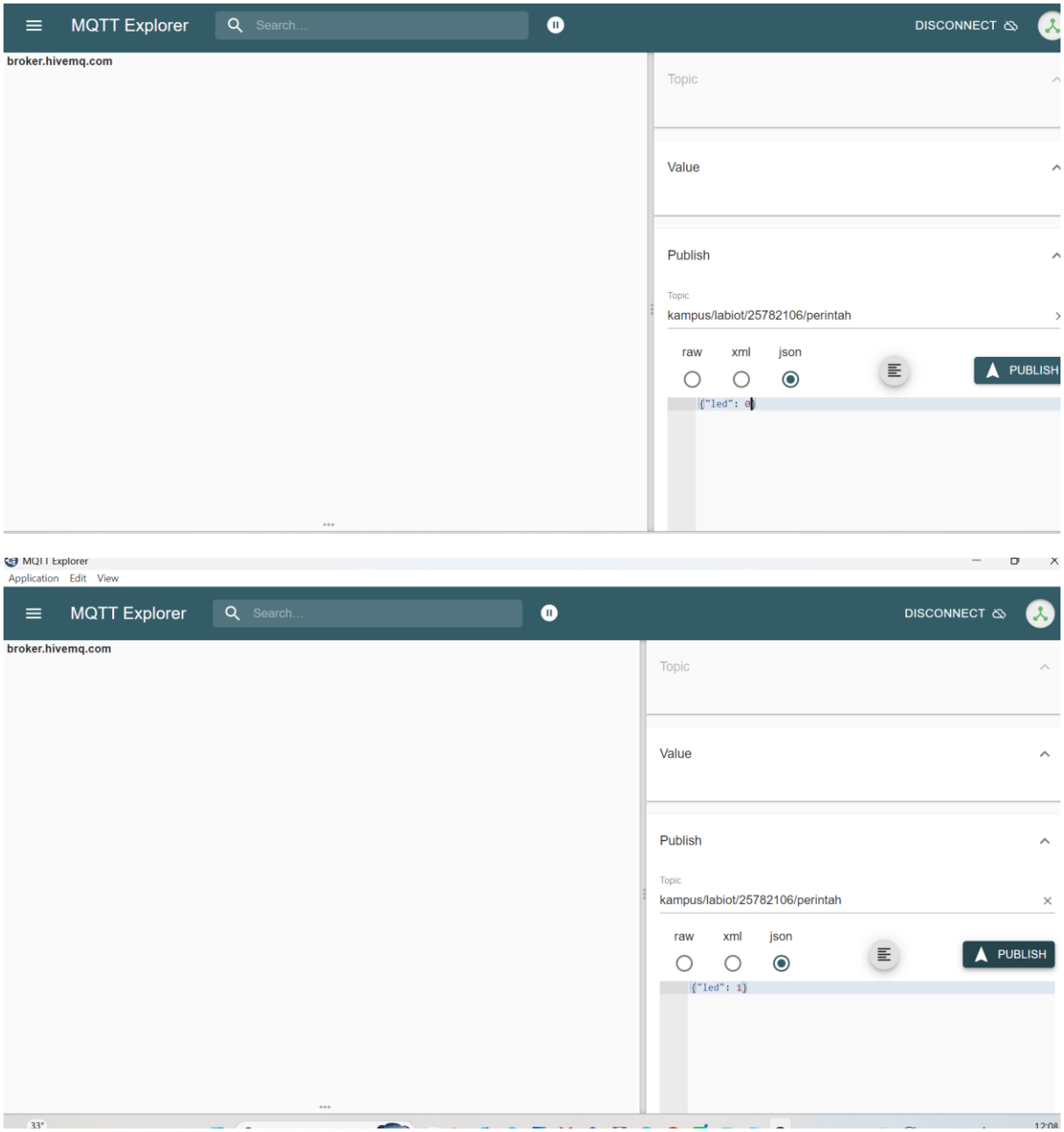






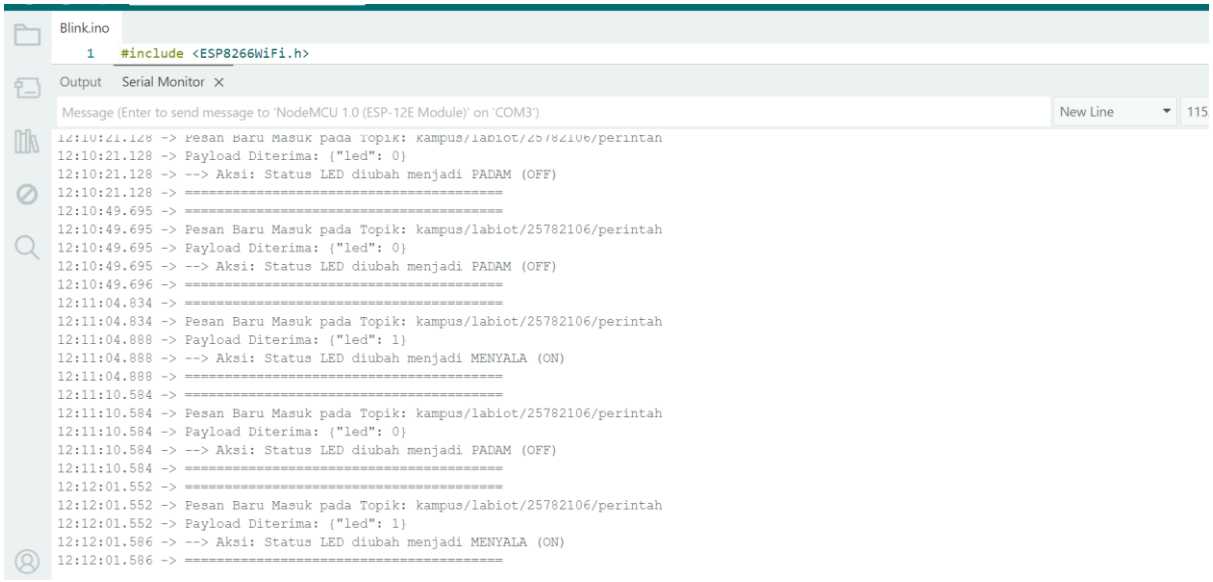
2. Pengujian Pengiriman Perintah melalui MQTT Explorer

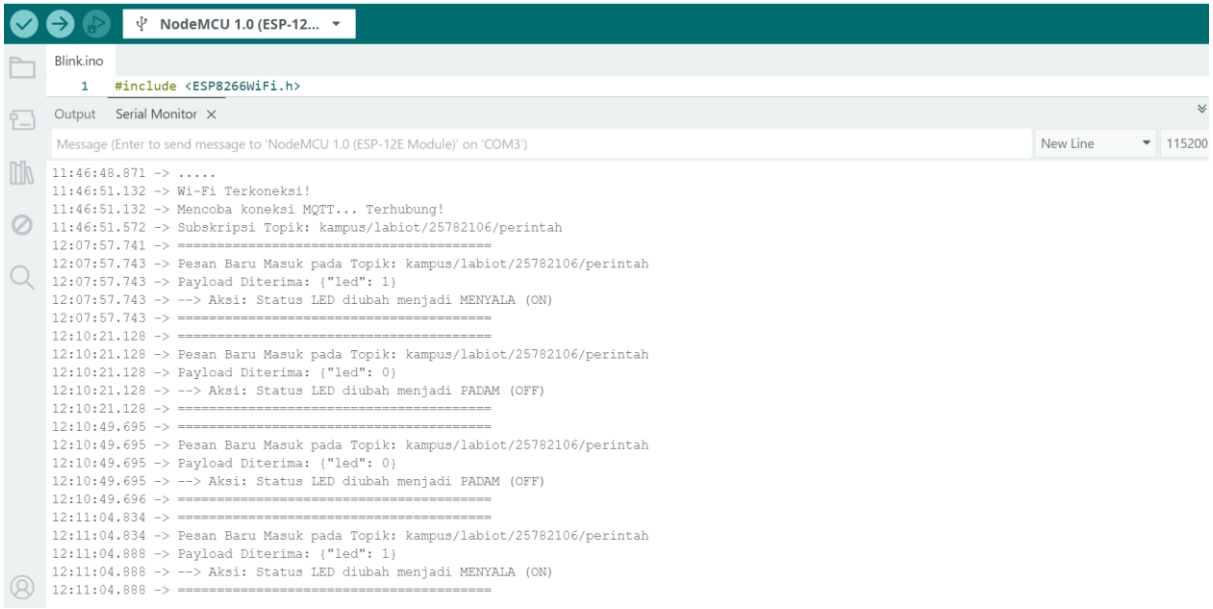
MQTT Explorer digunakan untuk mengirim perintah ke ESP8266 melalui broker MQTT. Pada pengujian ini, payload {"led":1} digunakan untuk menyalakan LED, sedangkan {"led":0} digunakan untuk mematikannya.



3. Hasil Serial Monitor saat Menerima Perintah Kendali LED

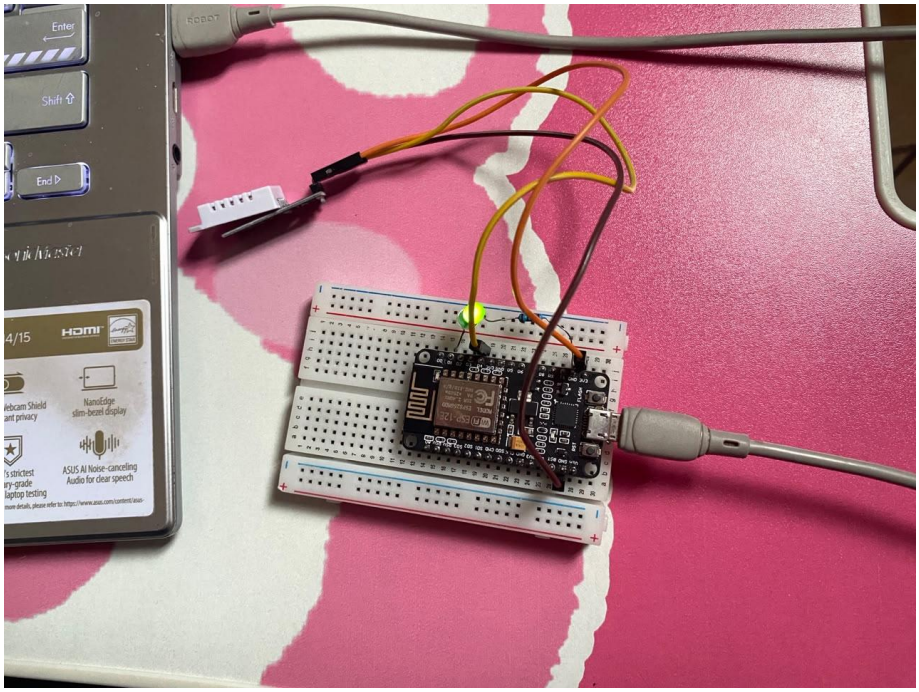
Serial Monitor menampilkan pesan yang diterima dari broker MQTT, termasuk topic dan payload perintah. ESP8266 kemudian memproses perintah tersebut untuk mengubah kondisi LED sesuai dengan nilai yang dikirim.

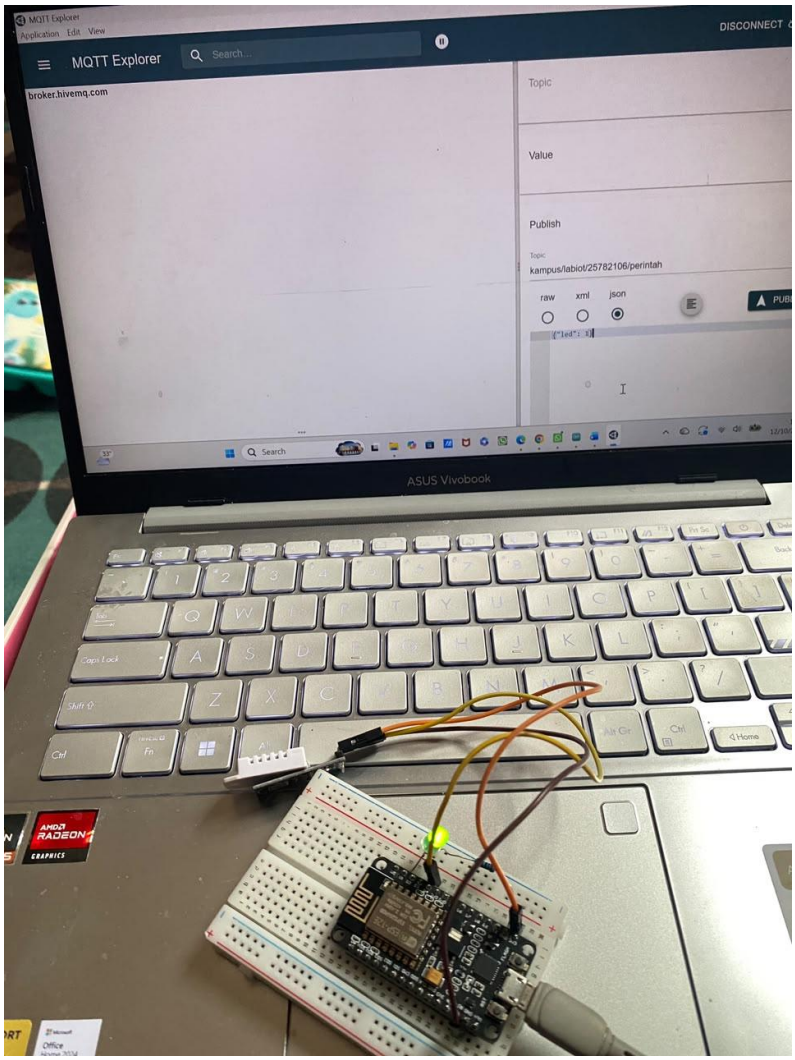




4. Rangkaian ketika LED On

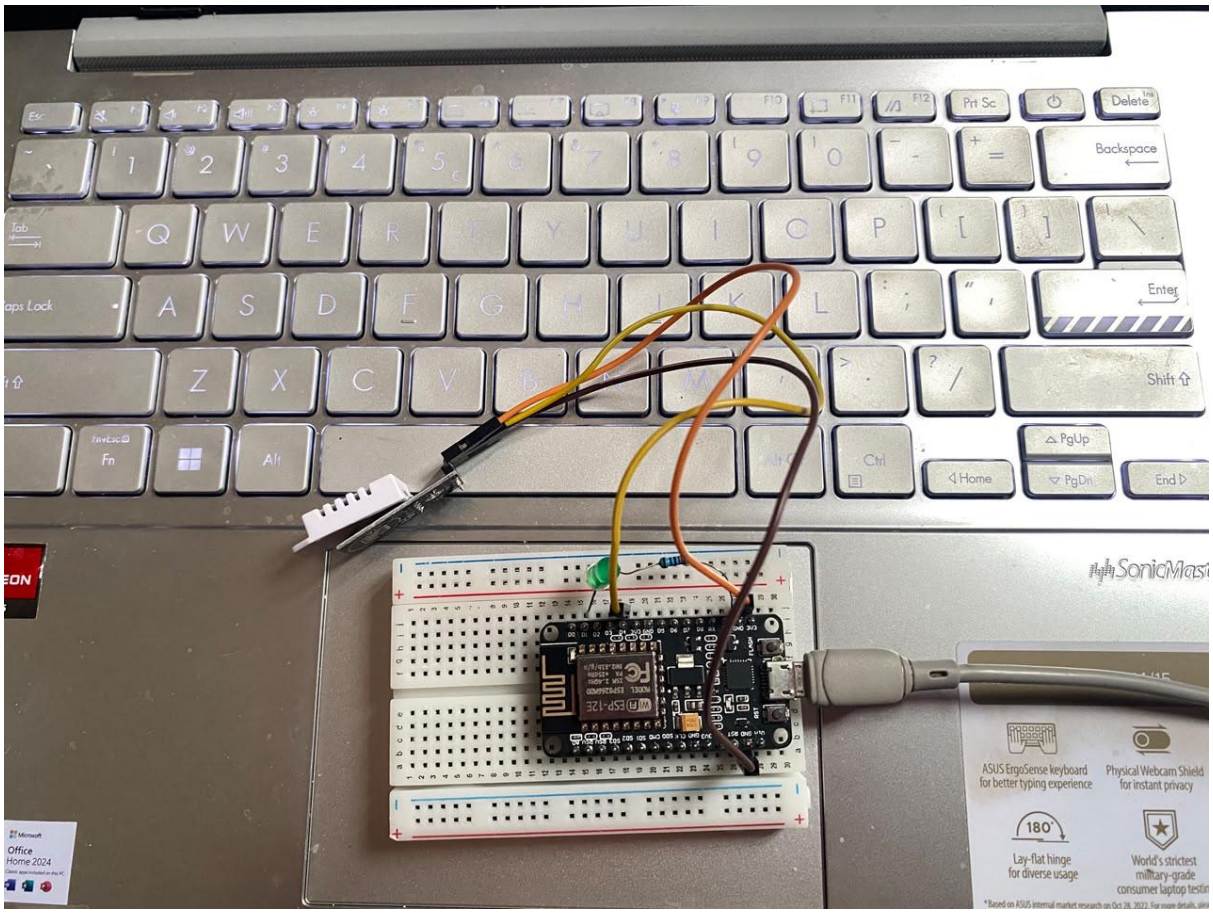
Gambar menunjukkan pengujian LED pada rangkaian NodeMCU ESP8266. Ketika menerima perintah {"led":1}, LED seharusnya menyala, sedangkan perintah {"led":0} digunakan untuk mematikannya. Dokumentasi ini menunjukkan pengujian kendali LED melalui MQTT.

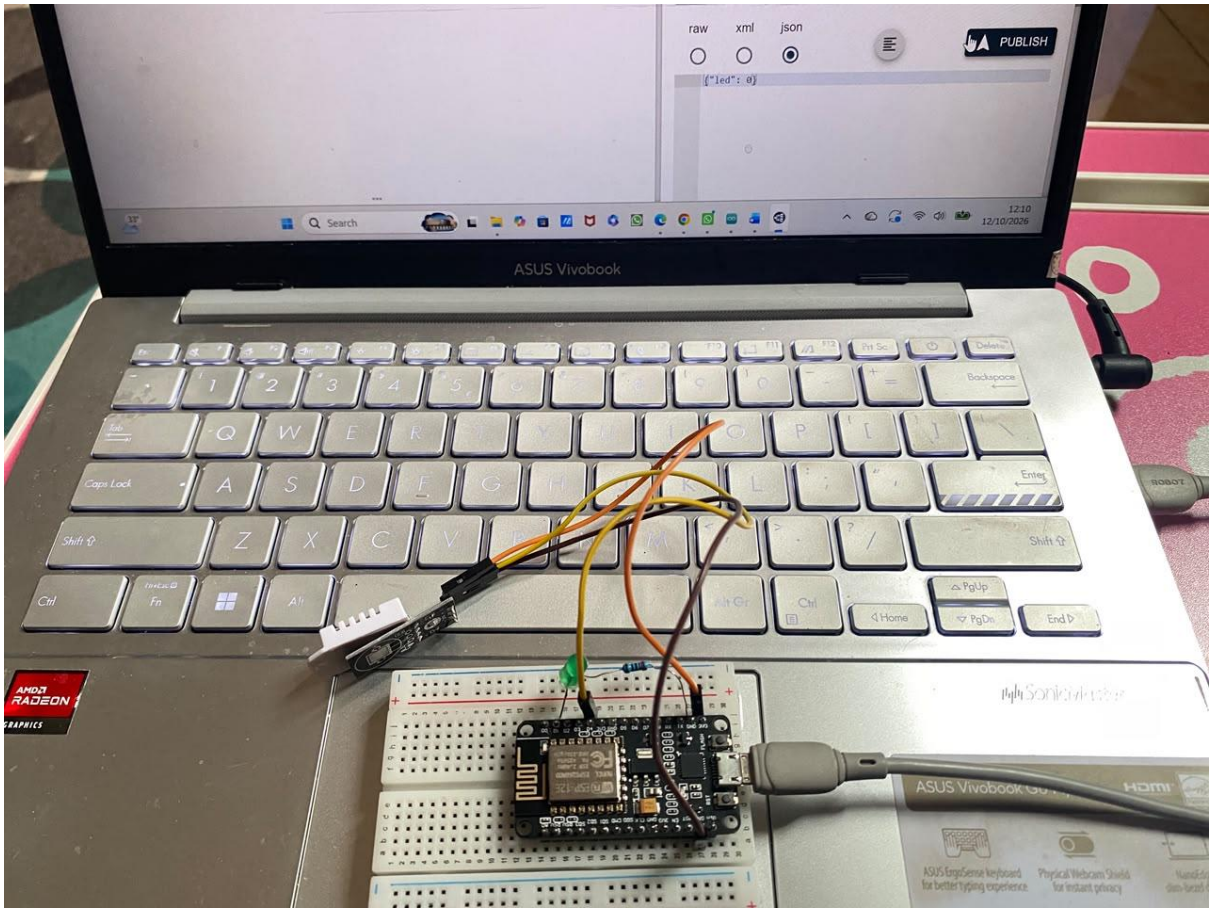




5. Rangkaian ketika LED OFF

Pada gambar ini, LED terlihat dalam kondisi mati setelah menerima perintah dengan payload {"led":0}. Hal ini menunjukkan bahwa perintah melalui MQTT dapat digunakan untuk mengendalikan LED

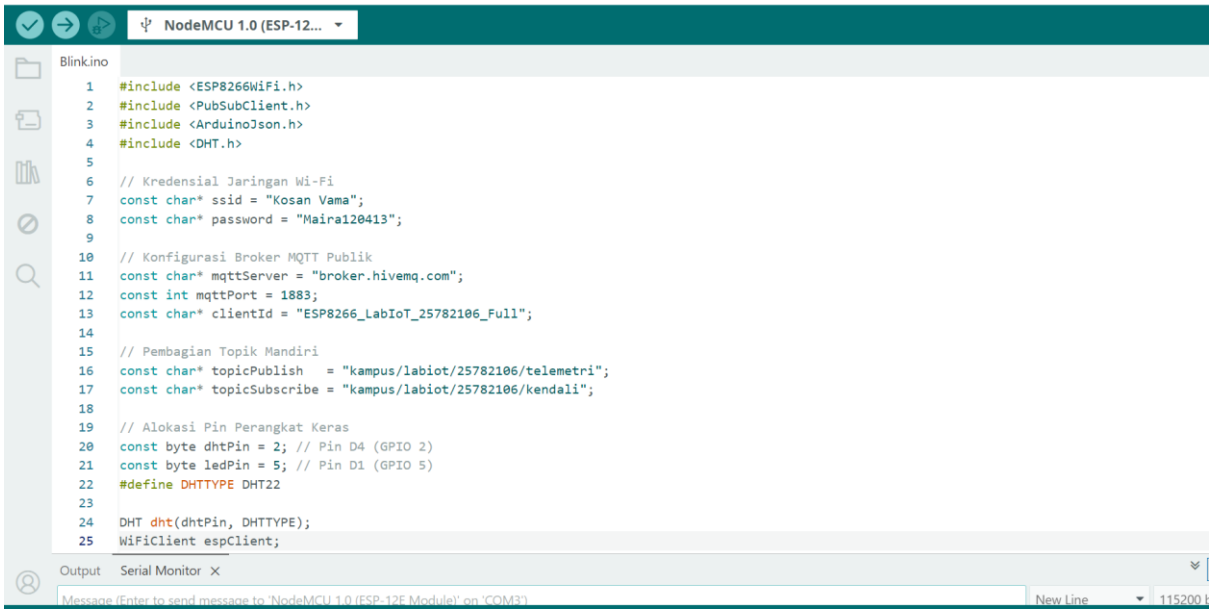




6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

1. Kode Program

Kode program menggabungkan pengiriman data sensor dan penerimaan perintah kendali LED. ESP8266 mengirim data suhu dan kelembapan secara berkala, sekaligus menerima perintah dari broker MQTT tanpa harus menghentikan proses telemetri.

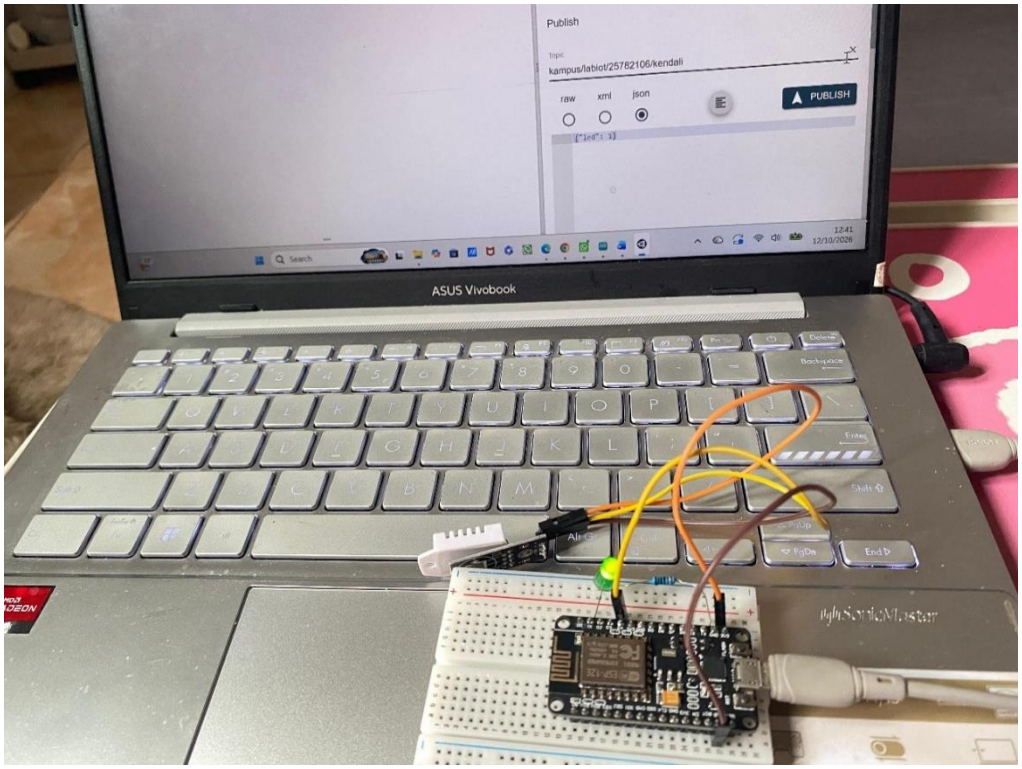


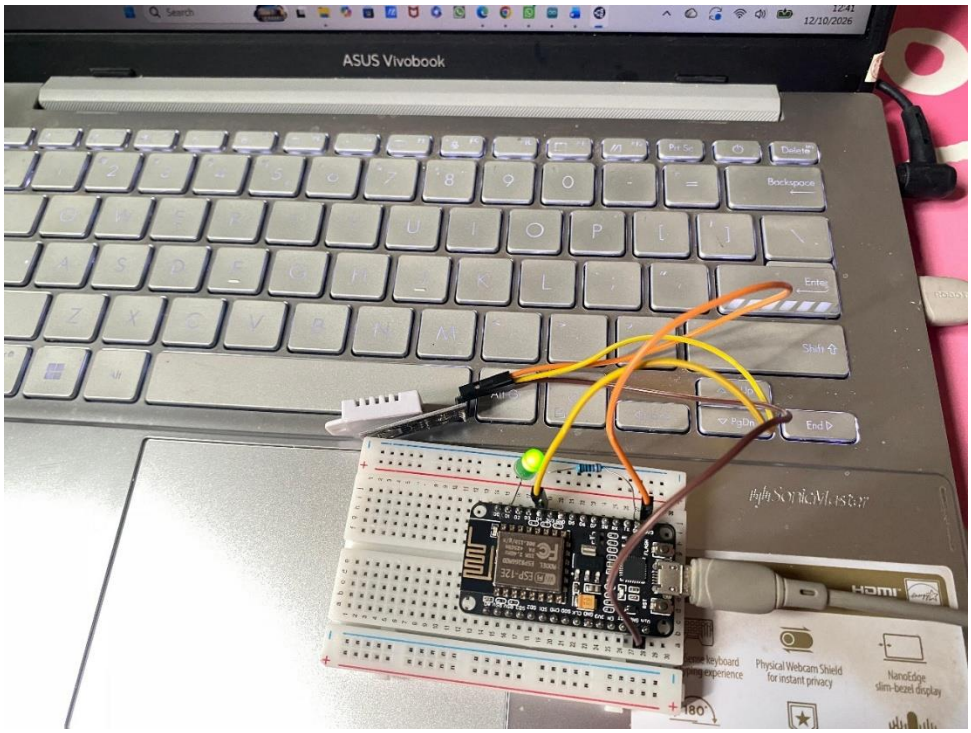


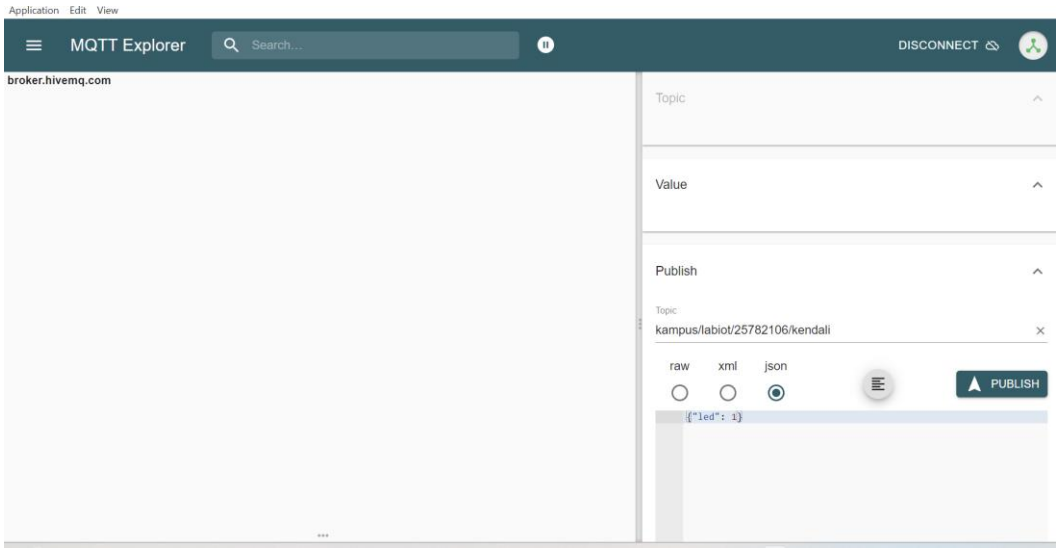


2. Rangkaian

Rangkaian menggabungkan sensor DHT dan LED dengan NodeMCU ESP8266. Sensor digunakan untuk membaca kondisi lingkungan, sedangkan LED berfungsi sebagai indikator yang dapat dikendalikan melalui MQTT.

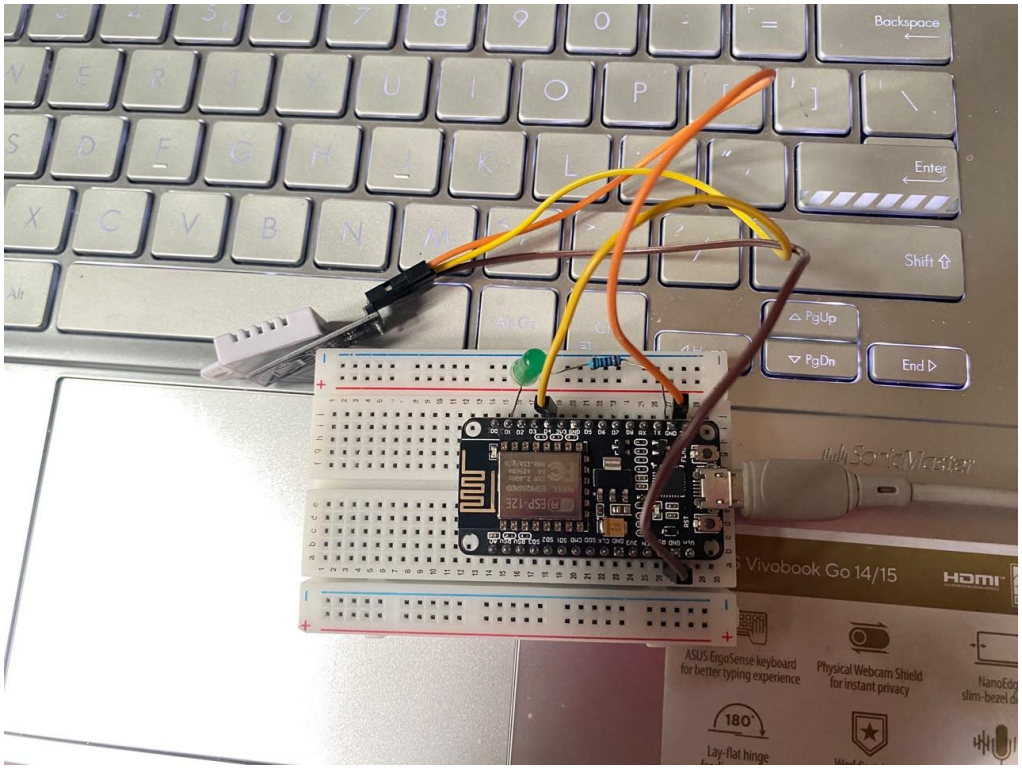

Publish
Topic
kampus/labiot/25782106/kendali
raw
xml
json
PUBLISH
ASUS Vivobook

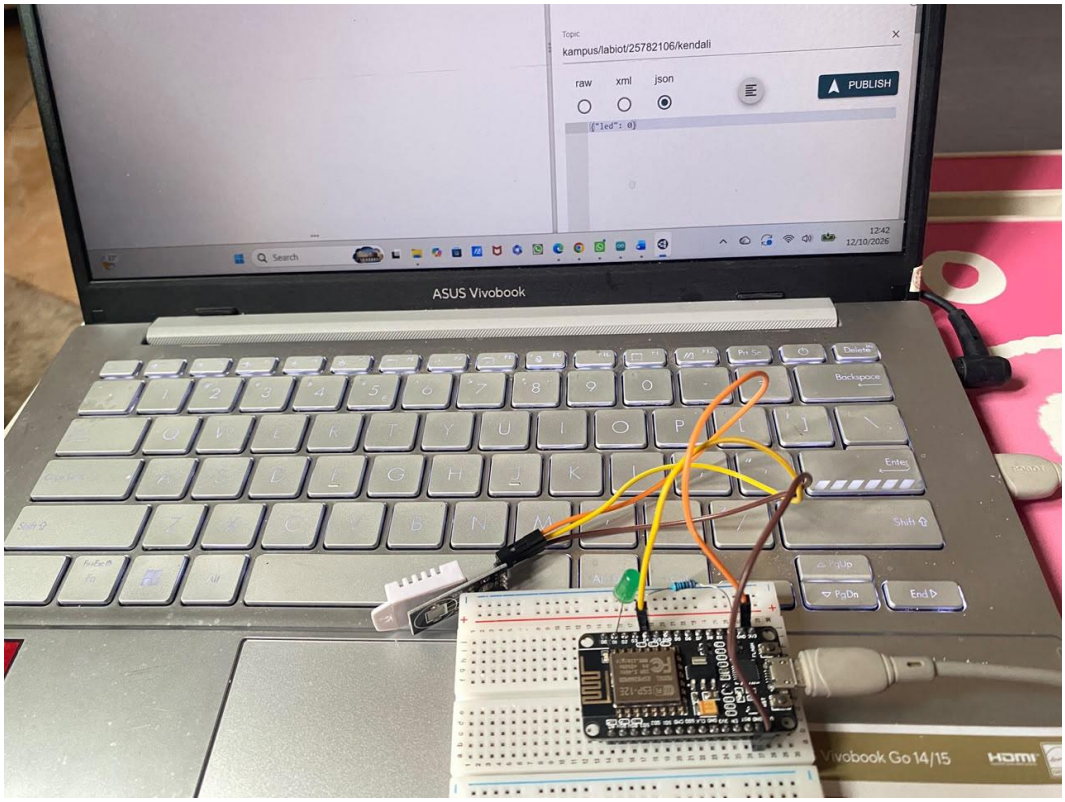

ASUS Vivobook
Physical Webcam Shield for instant privacy
NanoEdge slim-bezel display

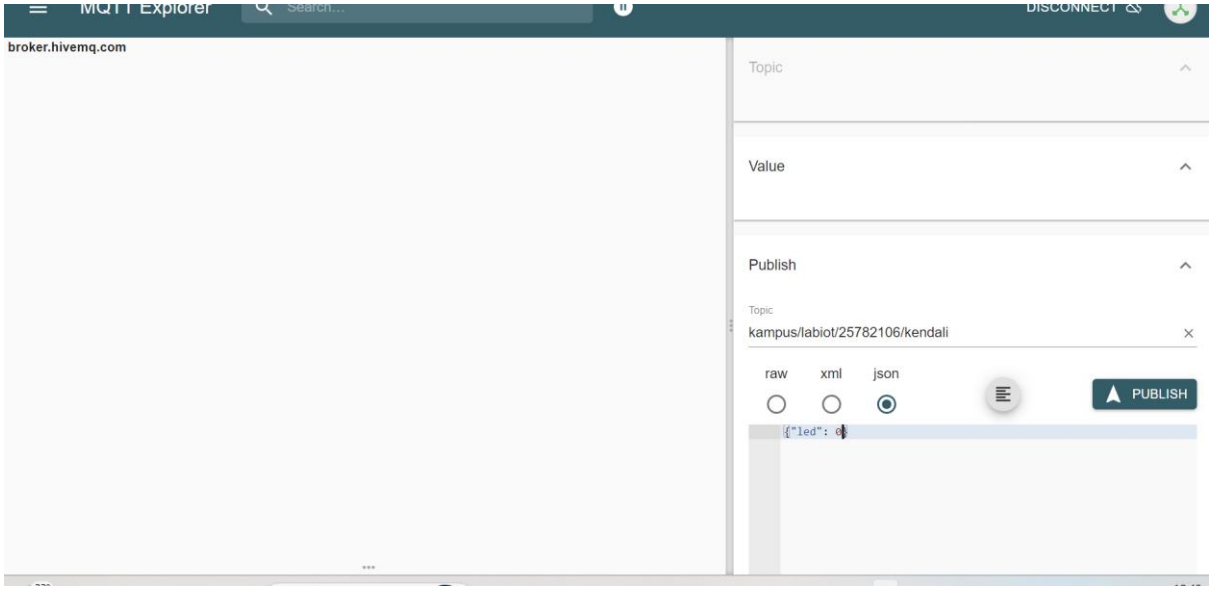


KETIKA LED OFF

Gambar menunjukkan kondisi LED saat tidak menyala. Kondisi ini sesuai dengan status LED mati pada program sebelum atau setelah menerima perintah untuk mematikannya.

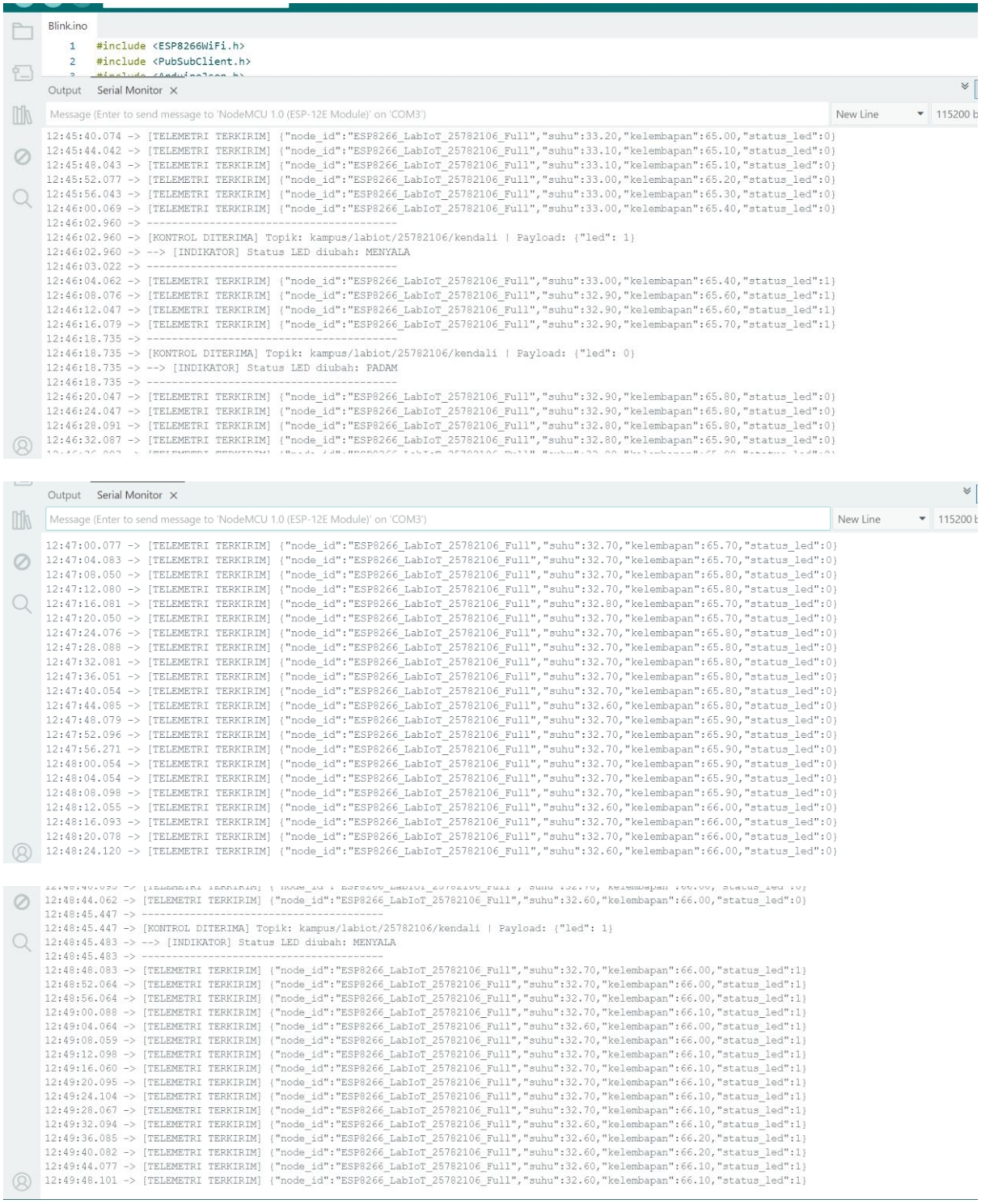






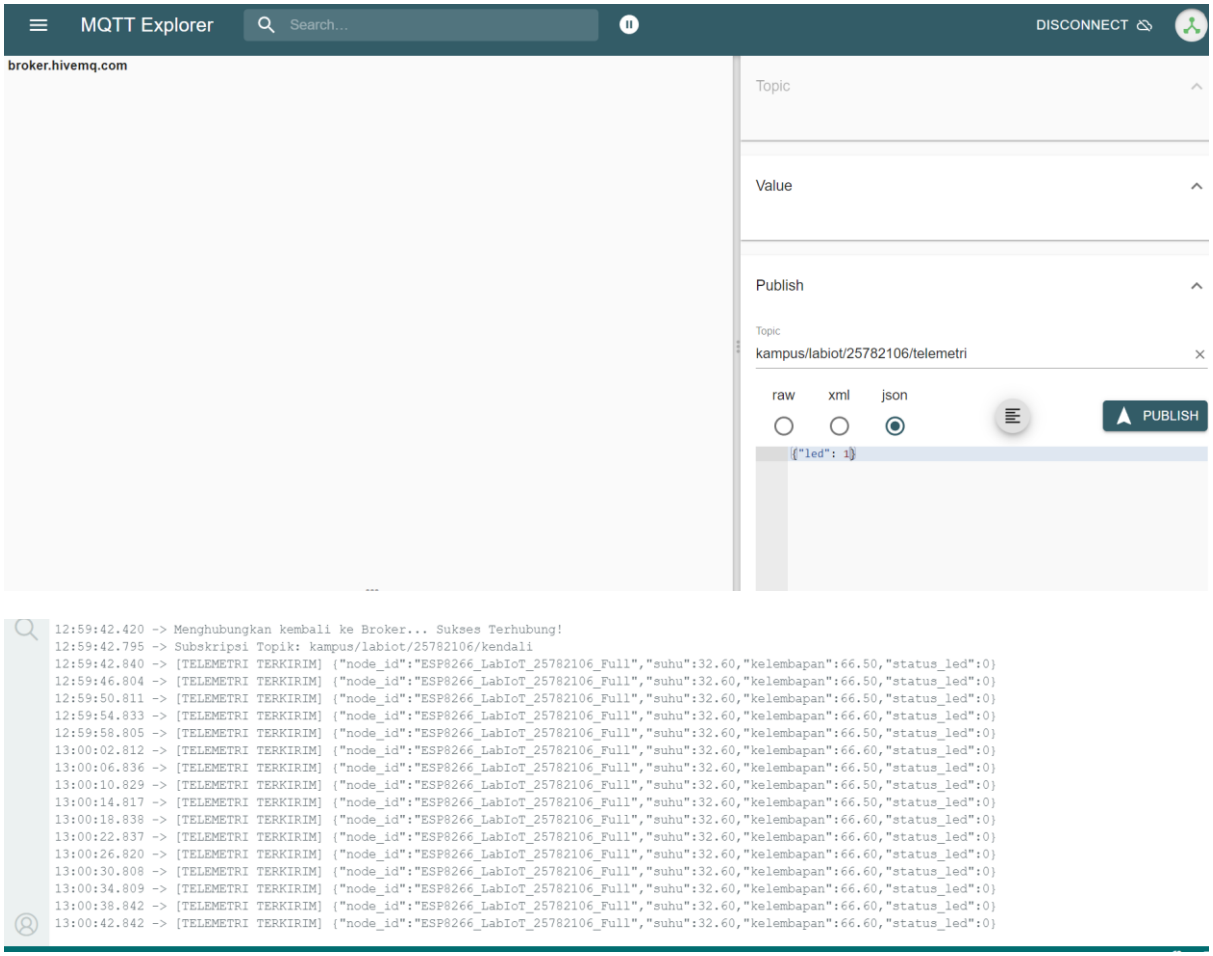
3. Hasil Serial Monitor

Serial Monitor menampilkan proses komunikasi MQTT, mulai dari koneksi hingga pengiriman data telemetri dan penerimaan perintah kendali. Data sensor dikemas dalam format JSON dan dikirim ke broker, sedangkan perintah yang masuk diproses untuk mengubah kondisi LED.

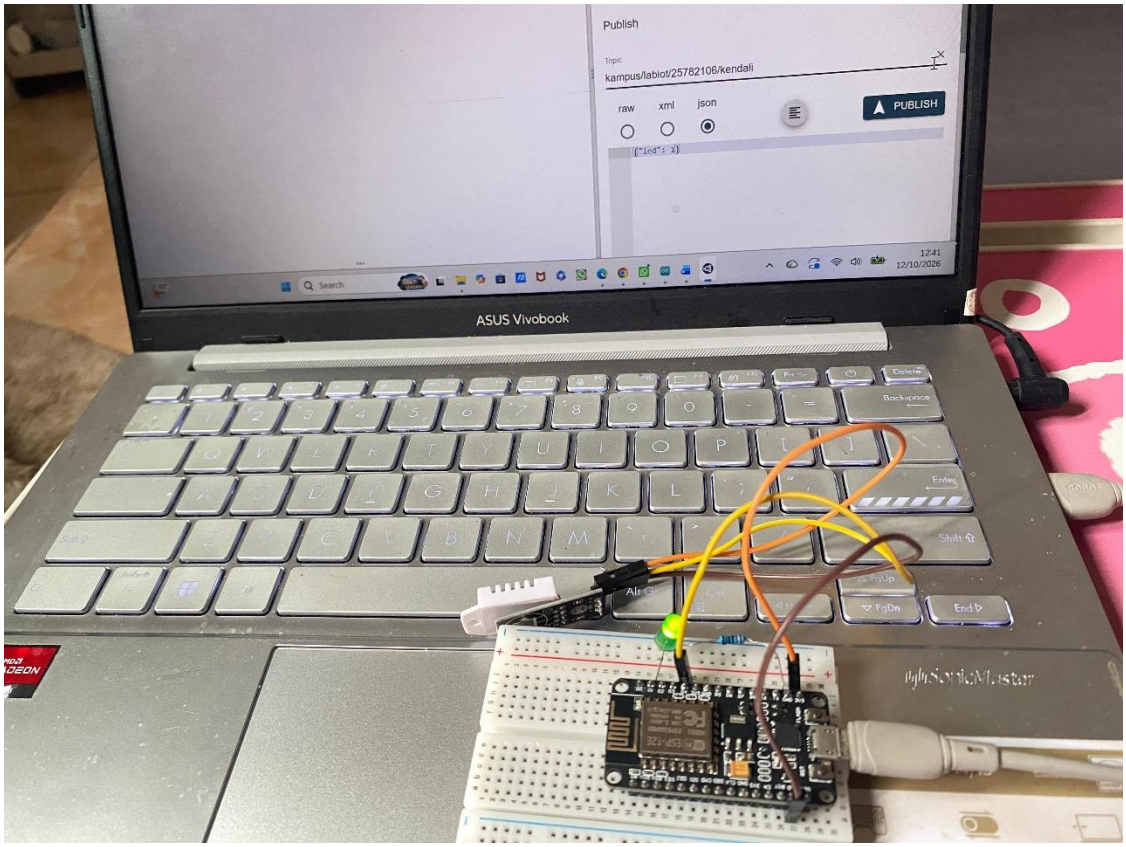


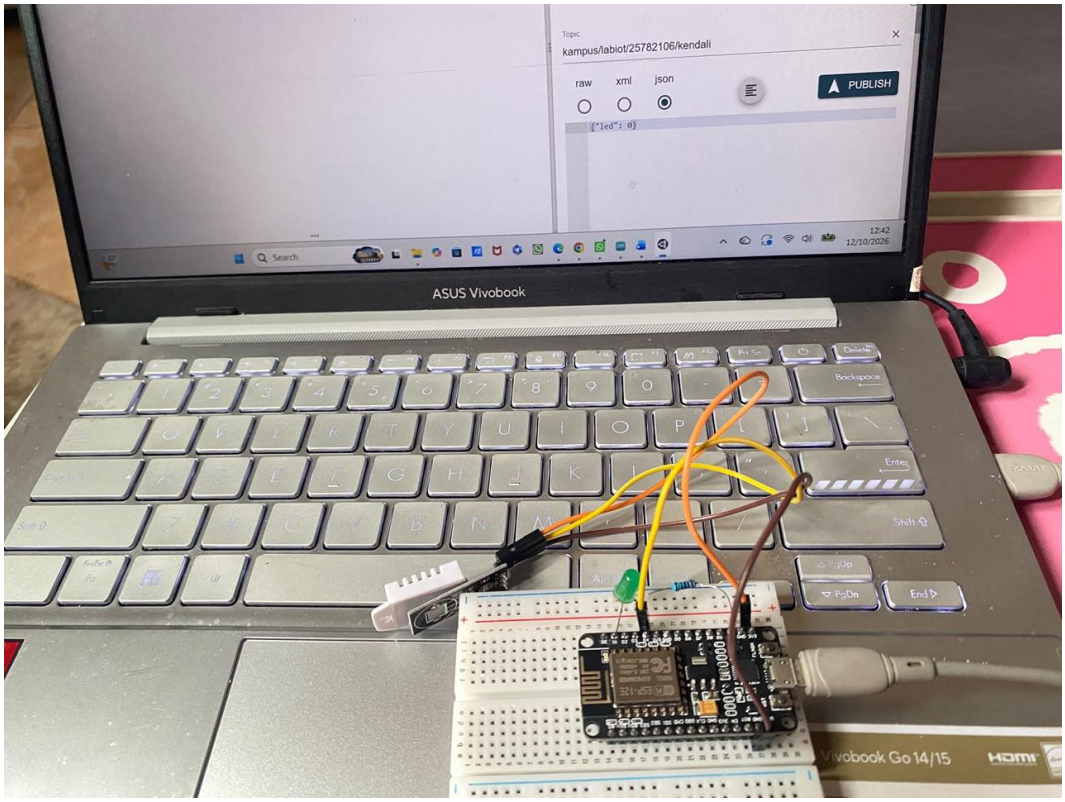
4. Hasil telemetri pada MQTT Explorer

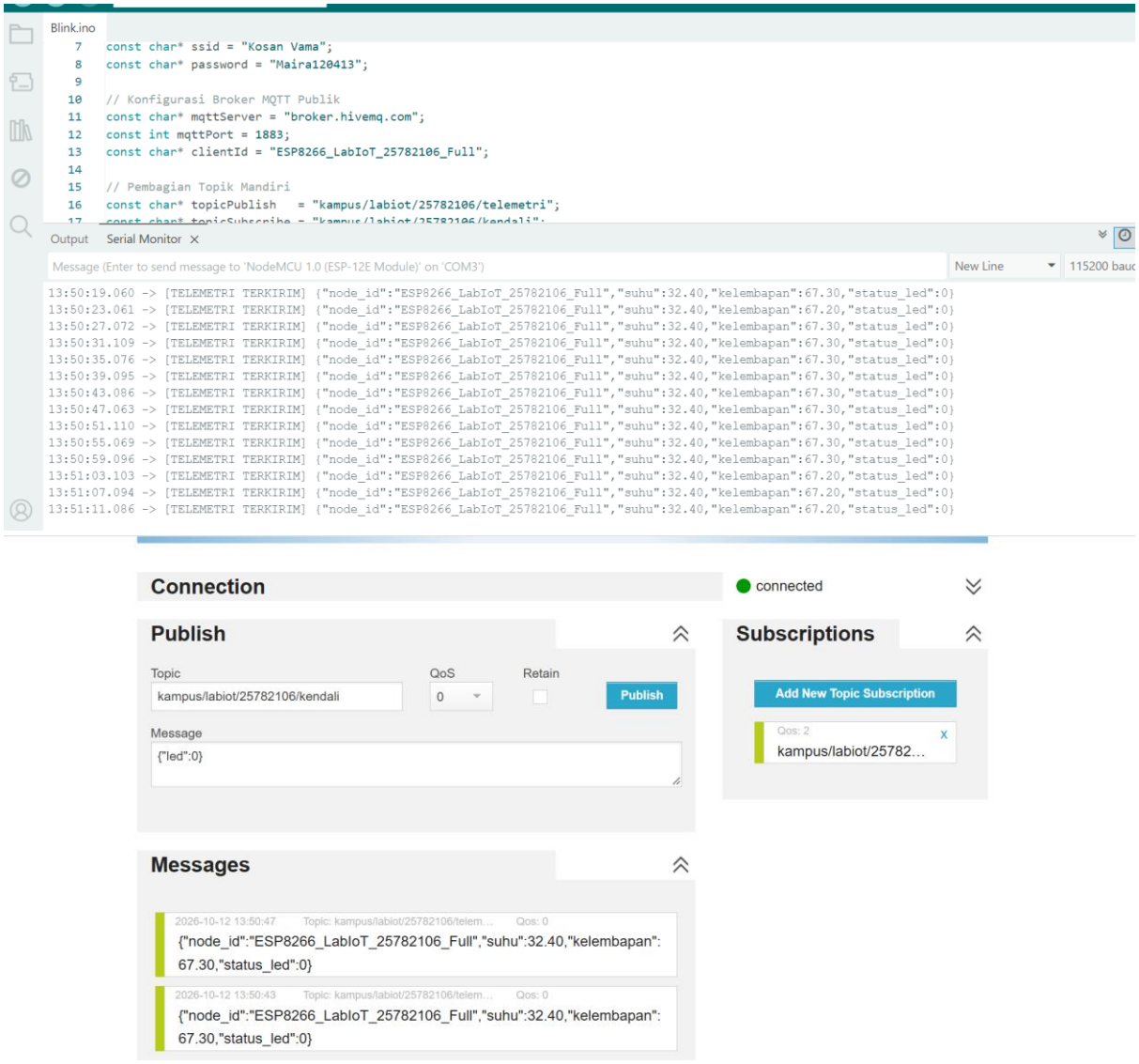
Data telemetri dapat dipantau melalui topic yang sudah ditentukan. Informasi seperti suhu, kelembapan, dan status LED dikirim dalam format JSON sehingga lebih mudah dibaca dan dipantau melalui MQTT Explorer.



6.5.4 Pengujian Diagnostik dengan MQTT Explorer


Publish
Topic
kampus/labiot/25782106/kendali
raw
xml
json
PUBLISH
{"led": 1}
ASUS Vivobook


Topic
kampus/labiot/25782106/kendali
raw
xml
json
PUBLISH
{"led": 0}
ASUS Vivobook
Vivobook Go 14/15



6.6 Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):
Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.

Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.

Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).

Jawaban:

1. Efisiensi penggunaan bandwidth pada paket data seluler

Menurut saya, MQTT lebih efisien dalam penggunaan bandwidth karena ukuran pesan dan informasi tambahan yang dikirim relatif kecil. MQTT menggunakan pola publish-subscribe, sehingga perangkat cukup mengirim data ke broker tanpa harus membuat permintaan HTTP berulang kali untuk setiap pembacaan sensor. Koneksi MQTT juga bisa dipertahankan agar perangkat tidak perlu membangun koneksi baru untuk setiap pengiriman data.

Sementara itu, HTTP biasanya membawa informasi tambahan berupa header pada setiap permintaan dan respons. Jika sensor mengirim data berkali-kali, informasi tambahan tersebut dapat membuat penggunaan bandwidth lebih besar. Karena itu, MQTT cocok untuk perangkat IoT yang mengirim data kecil secara berkala melalui jaringan seluler.

2. Beban komputasi mikrokontroler (CPU overhead) saat menangani pertukaran data dua arah

MQTT dapat mengurangi beban pemrosesan mikrokontroler karena protokol ini dirancang untuk komunikasi yang ringan dan menggunakan koneksi yang dapat dipertahankan. Pada program NodeMCU, fungsi client.loop() digunakan untuk memelihara koneksi MQTT sekaligus memproses pesan masuk. Ketika pesan kendali diterima pada topik yang sudah di-subscribe, fungsi callback akan menangani pesan tersebut.

Sebaliknya, komunikasi HTTP umumnya menggunakan pola permintaan dan respons. Jika perangkat perlu terus membaca sensor sekaligus menerima perintah, program harus mengatur proses permintaan dan respons tersebut. Dengan MQTT, proses pengiriman telemetri dan penerimaan perintah bisa berjalan melalui satu koneksi MQTT, sehingga lebih praktis untuk sistem dua arah. Namun, beban CPU yang sebenarnya tetap bergantung pada implementasi program dan kondisi jaringan.

3. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (1-to-N distribution)

MQTT lebih praktis untuk mengirim data ke banyak penerima karena menggunakan broker sebagai pusat distribusi pesan. Perangkat pengirim cukup melakukan publish ke suatu topik. Selanjutnya, broker meneruskan pesan tersebut kepada semua klien yang sudah melakukan subscribe ke topik yang sesuai.

Contohnya, NodeMCU mengirim data suhu ke topik kampus/labiot/25782106/telemetri. Beberapa perangkat, seperti laptop operator dan dashboard pemantauan, dapat berlangganan ke topik tersebut untuk menerima data yang sama. NodeMCU tidak perlu mengirim pesan satu per satu kepada setiap penerima. Pada HTTP, fungsi serupa biasanya memerlukan mekanisme tambahan, misalnya server yang mendistribusikan data atau permintaan terpisah dari masing-masing penerima.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):
Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

Memantau parameter suhu dari seluruh mesin di semua lini produksi.

Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.

Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

Jawab:

1. Untuk menerima data suhu dari semua mesin di semua lini produksi, pola subscription yang digunakan adalah:

pabrik/+/+/suhu

Tanda + digunakan untuk menggantikan satu tingkat nama topik. Jadi, pola ini dapat menerima data suhu dari berbagai lini produksi dan mesin, tetapi hanya untuk topik suhu.

2. Untuk menerima semua jenis data sensor dari mesin M1 di lini produksi 2, pola subscription yang digunakan adalah:

pabrik/line2/m1/+

Tanda + digunakan untuk menerima semua jenis data sensor pada mesin M1 di lini produksi 2, misalnya suhu, kelembapan, dan tekanan.

3. Untuk menerima semua pesan dari seluruh topik yang berada di bawah pabrik, pola subscription yang digunakan adalah:

pabrik/#

Tanda # digunakan untuk menerima semua topik dan subtopik di bawah pabrik, termasuk topik pada tingkat yang lebih dalam.

Wildcard + digunakan untuk menggantikan satu tingkat nama topik, sedangkan wildcard # digunakan untuk menerima semua tingkat topik setelah posisi tersebut. Wildcard # harus diletakkan di bagian akhir pola subscription.

Latihan 3 (Implementasi Serialisasi Data Dinamis):
Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

"uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).

"ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()). Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

Jawab:

Pada latihan ini, saya menambahkan dua informasi baru pada data telemetri MQTT, yaitu uptime_detik dan ip_lokal.

Informasi uptime_detik digunakan untuk mengetahui berapa lama NodeMCU ESP8266 telah berjalan sejak program dimulai. Nilai ini diperoleh dari fungsi millis() yang dibagi 1000 agar hasilnya dalam satuan detik.

Sementara itu, ip_lokal digunakan untuk menampilkan alamat IP lokal yang diperoleh NodeMCU ketika terhubung ke jaringan Wi-Fi. Informasi ini diperoleh menggunakan WiFi.localIP().toString().

Setelah program diunggah, kedua informasi tersebut berhasil muncul pada data JSON di Serial Monitor dan diterima melalui HiveMQ WebSocket Client pada topik kampus/labiot/25782106/telemetri. Dengan demikian, penambahan informasi waktu aktif perangkat dan alamat IP lokal berhasil dilakukan.

Kode yang di tambahkan



Hasil serial monitor :



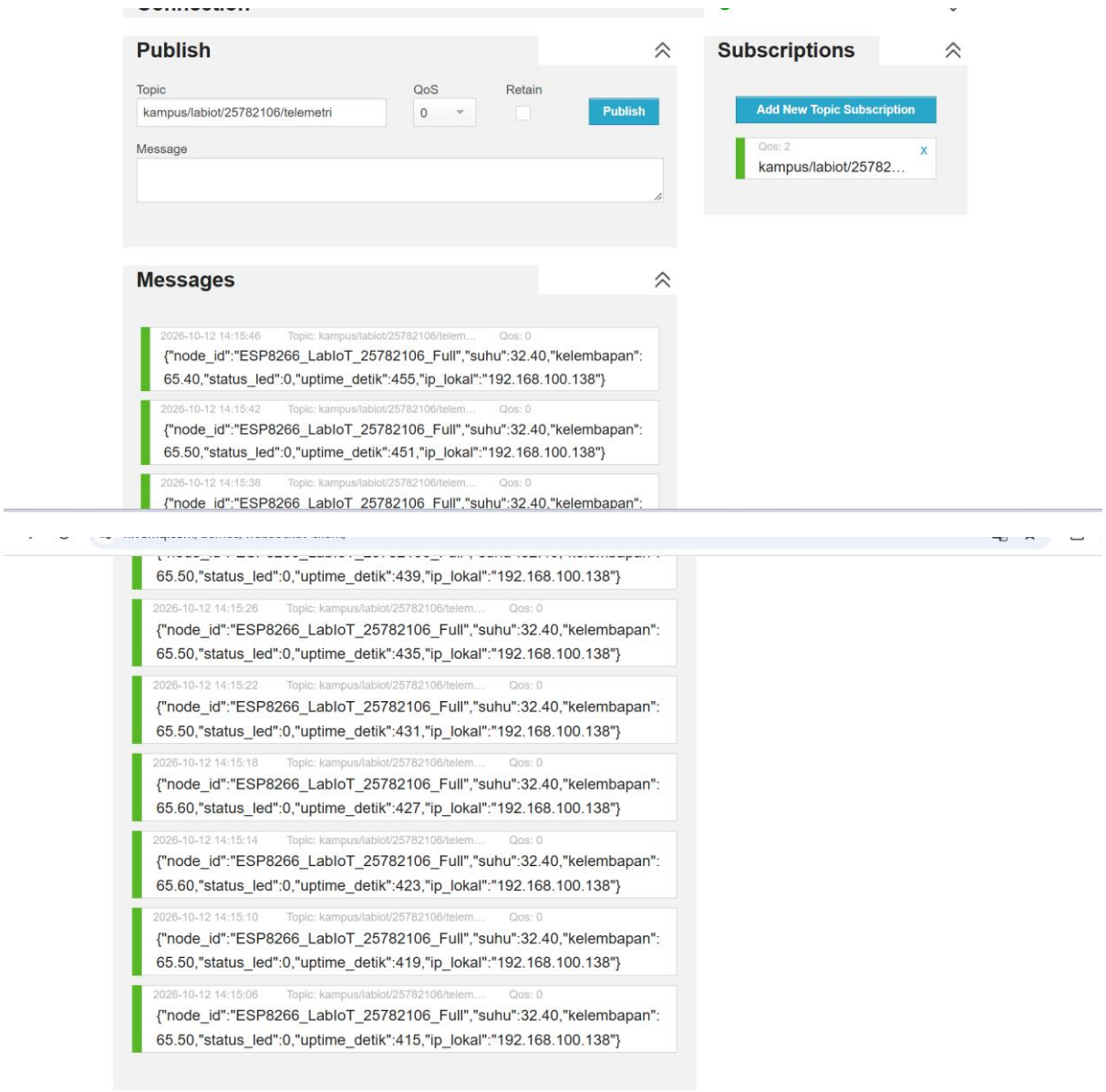
```
101   float h = dht.readHumidity();
102
103   if (!isnan(t) && !isnan(h)) {
```

Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3') New Line 115200 baud

```
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.50,"kelembapan":65.60,"status_led":0,"uptime_detik":175,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.50,"kelembapan":65.60,"status_led":0,"uptime_detik":179,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.50,"kelembapan":65.50,"status_led":0,"uptime_detik":183,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.50,"kelembapan":65.60,"status_led":0,"uptime_detik":187,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.50,"kelembapan":65.60,"status_led":0,"uptime_detik":191,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.50,"kelembapan":65.50,"status_led":0,"uptime_detik":195,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.50,"kelembapan":65.60,"status_led":0,"uptime_detik":199,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.50,"kelembapan":65.60,"status_led":0,"uptime_detik":203,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.50,"status_led":0,"uptime_detik":207,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.50,"status_led":0,"uptime_detik":211,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.60,"status_led":0,"uptime_detik":215,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.60,"status_led":0,"uptime_detik":219,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.60,"status_led":0,"uptime_detik":223,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.60,"status_led":0,"uptime_detik":227,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.60,"status_led":0,"uptime_detik":231,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.50,"status_led":0,"uptime_detik":235,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.60,"status_led":0,"uptime_detik":239,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.60,"status_led":0,"uptime_detik":243,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.60,"status_led":0,"uptime_detik":247,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.70,"status_led":0,"uptime_detik":251,"ip_lokal":"192.168.100.138"}
:LEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782106_Full","suhu":32.40,"kelembapan":65.80,"status_led":0,"uptime_detik":255,"ip_lokal":"192.168.100.138"}
```

Hasil di HiveMQ WebSocket Client



Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):
Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?

Jawab:

Jika delay(6000) ditambahkan ke dalam fungsi loop(), program akan berhenti menjalankan instruksi berikutnya selama kurang lebih 6 detik. Akibatnya, pemanggilan client.loop() ikut tertunda sehingga pesan MQTT yang masuk tidak bisa langsung diproses dan fungsi callback menjadi kurang responsif.

Contohnya, ketika operator mengirim perintah {"led": 1} untuk menyalakan LED, LED bisa terlambat menyala karena NodeMCU sedang menjalankan delay(). Hal ini membuat respons sistem menjadi lambat. Oleh karena itu, penggunaan delay() sebaiknya dihindari agar pengiriman telemetri dan penerimaan perintah dapat berjalan lebih responsif.

Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

Jawab:

Fungsi reconnect() digunakan untuk mencoba menghubungkan kembali NodeMCU ke broker MQTT ketika koneksi MQTT terputus. Program akan memeriksa status koneksi menggunakan client.connected(). Jika belum terhubung, program mencoba menyambung kembali melalui client.connect(clientId).

Jika koneksi berhasil, program akan menjalankan client.subscribe(topicSubscribe) agar NodeMCU kembali menerima pesan kendali dari topik yang sudah ditentukan. Namun, jika koneksi gagal, program menampilkan pesan kegagalan pada Serial Monitor, kemudian menunggu 4 detik sebelum mencoba kembali.

Dengan mekanisme ini, NodeMCU dapat melanjutkan komunikasi MQTT setelah koneksi berhasil dipulihkan. Jika yang terputus adalah Wi-Fi, proses penyambungan ulang Wi-Fi juga perlu ditangani agar perangkat dapat kembali terhubung ke broker.

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

1. Kode Program

Kode program digunakan untuk menggabungkan pemantauan suhu dan kelembapan dengan kendali LED otomatis dan manual. Program juga mengatur pengiriman data melalui MQTT serta penanganan kondisi tertentu berdasarkan hasil pembacaan sensor.

Tugas_6_7_Industrial_Safety.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// ================= KONFIGURASI WI-FI =================
const char* ssid = "Kosan Vama";
const char* password = "Maira120413";

// ================= KONFIGURASI MQTT ==================
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782106_Safety";

// ================= TOPIK MQTT ========================
const char* topicPublish =
  "kampus/labiot/25782106/telemetri";
const char* topicSubscribe =
  "kampus/labiot/25782106/kendali";
const char* topicAlert =
  "kampus/labiot/25782106/alert";
const char* topicStatus =
  "kampus/labiot/25782106/status";

// ================= PERANGKAT KERAS ===================
const byte dhtPin = 2;  // D4 / GPIO 2
const byte ledPin = 5;  // D1 / GPIO 5
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

// ================= PENGATURAN WAKTU ==================
const unsigned long telemetryInterval = 4000;
const unsigned long wifiRetryInterval = 5000;
const unsigned long mqttRetryInterval = 4000;

unsigned long lastTelemetryTime = 0;
unsigned long lastWiFiAttempt = 0;
unsigned long lastMqttAttempt = 0;

// ================= STATUS KESELAMATAN ================
bool overheat = false;
bool alertSent = false;
bool temperatureKnown = false;
int manualLed = LOW;

// ================= PESAN LAST WILL ===================
}

// ================= CALLBACK KENDALI ==================
void callback(char* topic, byte* payload, unsigned int length) {
  if (strcmp(topic, topicSubscribe) != 0) {
    return;
  }

  StaticJsonDocument<256> doc;
  DeserializationError err =
    deserializeJson(doc, payload, length);

  if (err) {
    Serial.println("[ERROR] JSON kendali tidak valid");
    return;
  }

  if (!doc.containsKey("led")) {
    return;
  }

  int val = doc["led"];

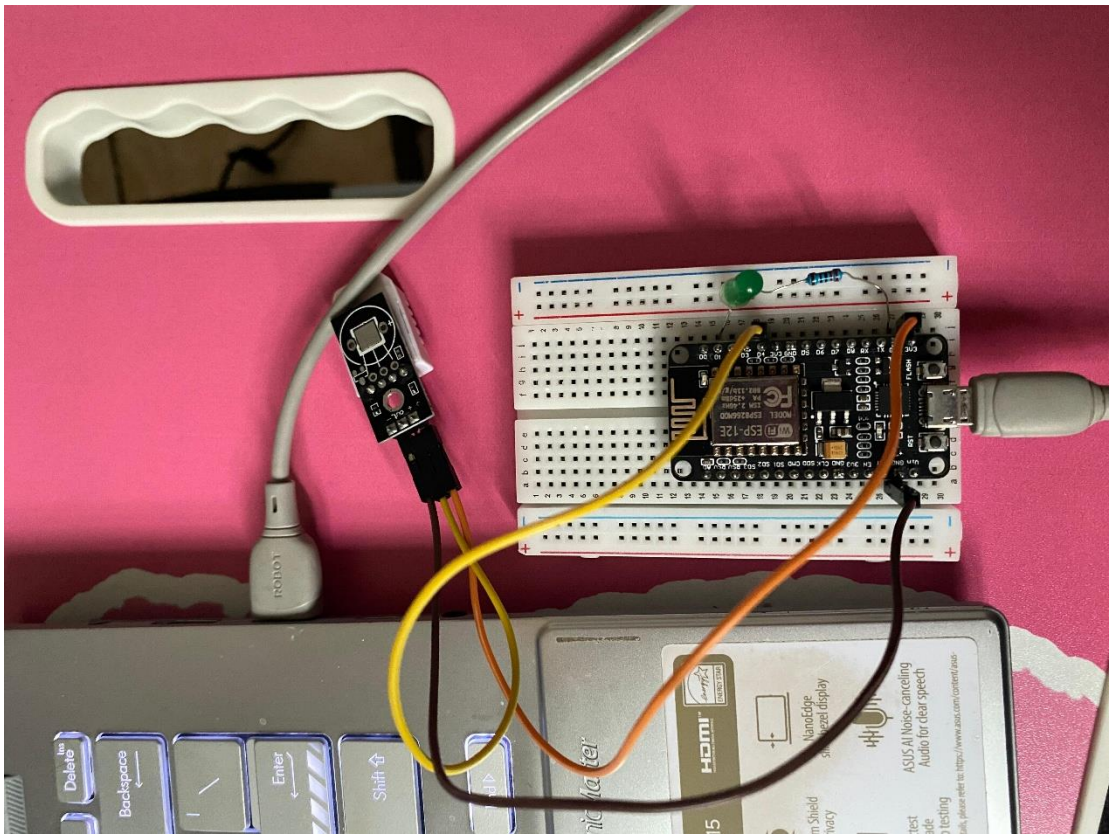
  if (val != 0 && val != 1) {
    Serial.println("[ERROR] Nilai LED harus 0 atau 1");
```

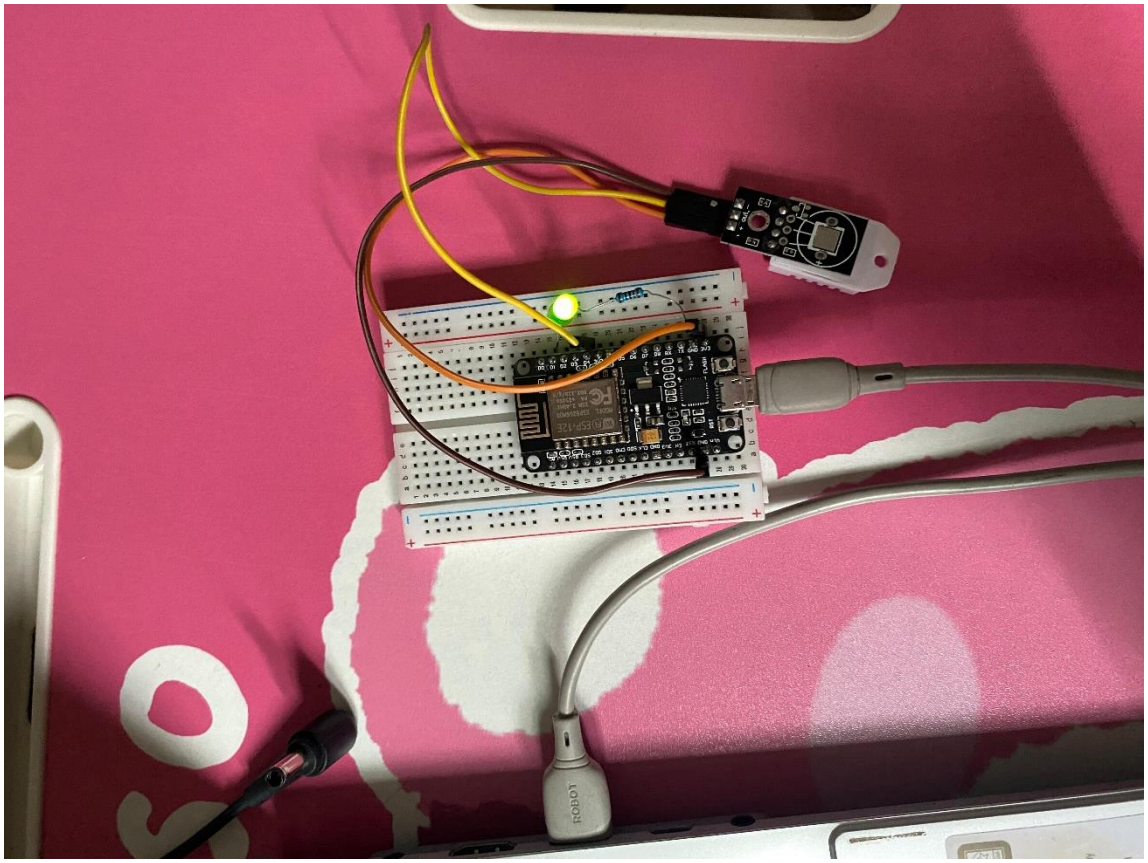
Tugas_6_7_Industrial_Safety.ino

```
251
252   // ===== MEMBUAT DATA TELEMETRI ======================
253   StaticJsonDocument<384> docOut;
254
255   docOut["node_id"] = clientId;
256   docOut["suhu"] = t;
257   docOut["kelembapan"] = h;
258   docOut["status_led"] = digitalRead(ledPin);
259   docOut["uptime_detik"] = millis() / 1000;
260   docOut["ip_lokal"] = WiFi.localIP().toString();
261   docOut["kondisi_suhu"] =
262     overheat ? "PANAS" : "NORMAL";
263
264   char buffer[384];
265   serializeJson(docOut, buffer);
266
267   // ===== KIRIM TELEMETRI: OUTPUT DUA BARIS ============
268   if (client.publish(topicPublish, buffer)) {
269     Serial.println("[TELEMETRI TERKIRIM]");
270     Serial.println(buffer);
271   } else {
272     Serial.println("[ERROR] Telemetri gagal dikirim");
273   }
274 }
275
```

2. Rangkaian Fisik

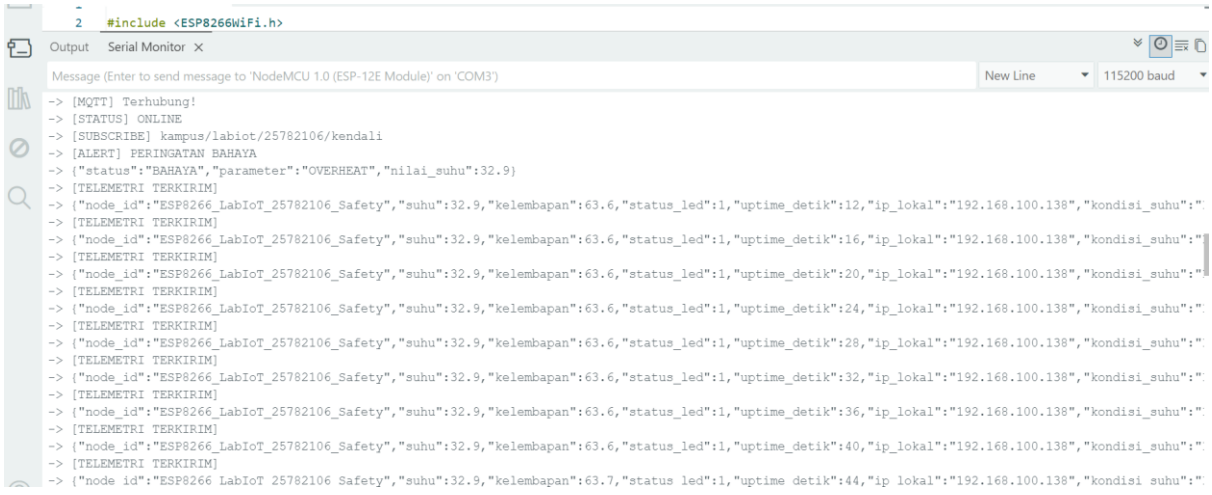
Rangkaian fisik menggunakan NodeMCU ESP8266, sensor DHT, dan LED yang terhubung sesuai pin pada program. Sensor digunakan untuk membaca kondisi lingkungan, sedangkan LED menjadi indikator ketika sistem menerima perintah atau mendeteksi kondisi tertentu.

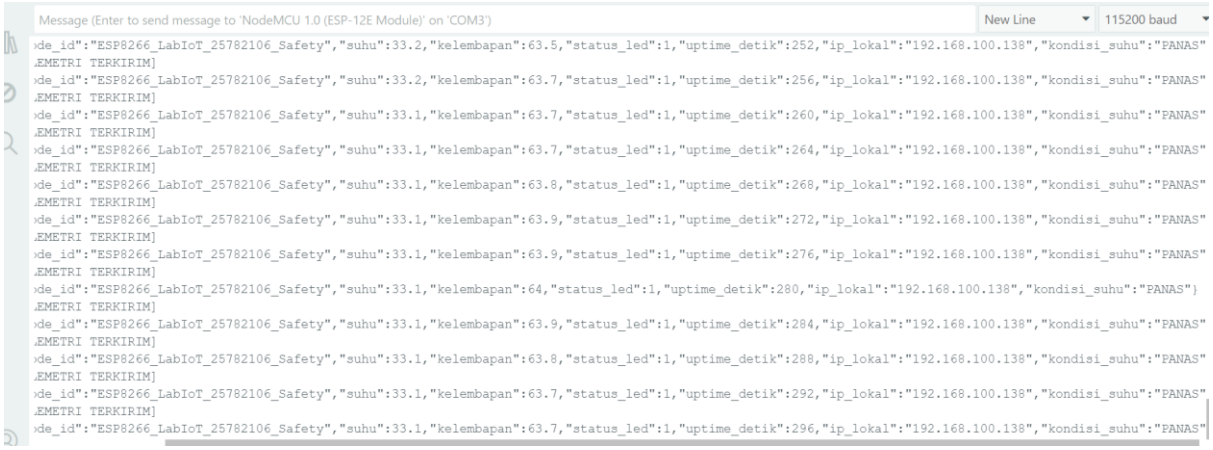




3. Hasil Serial Monitor ketika suhu panas

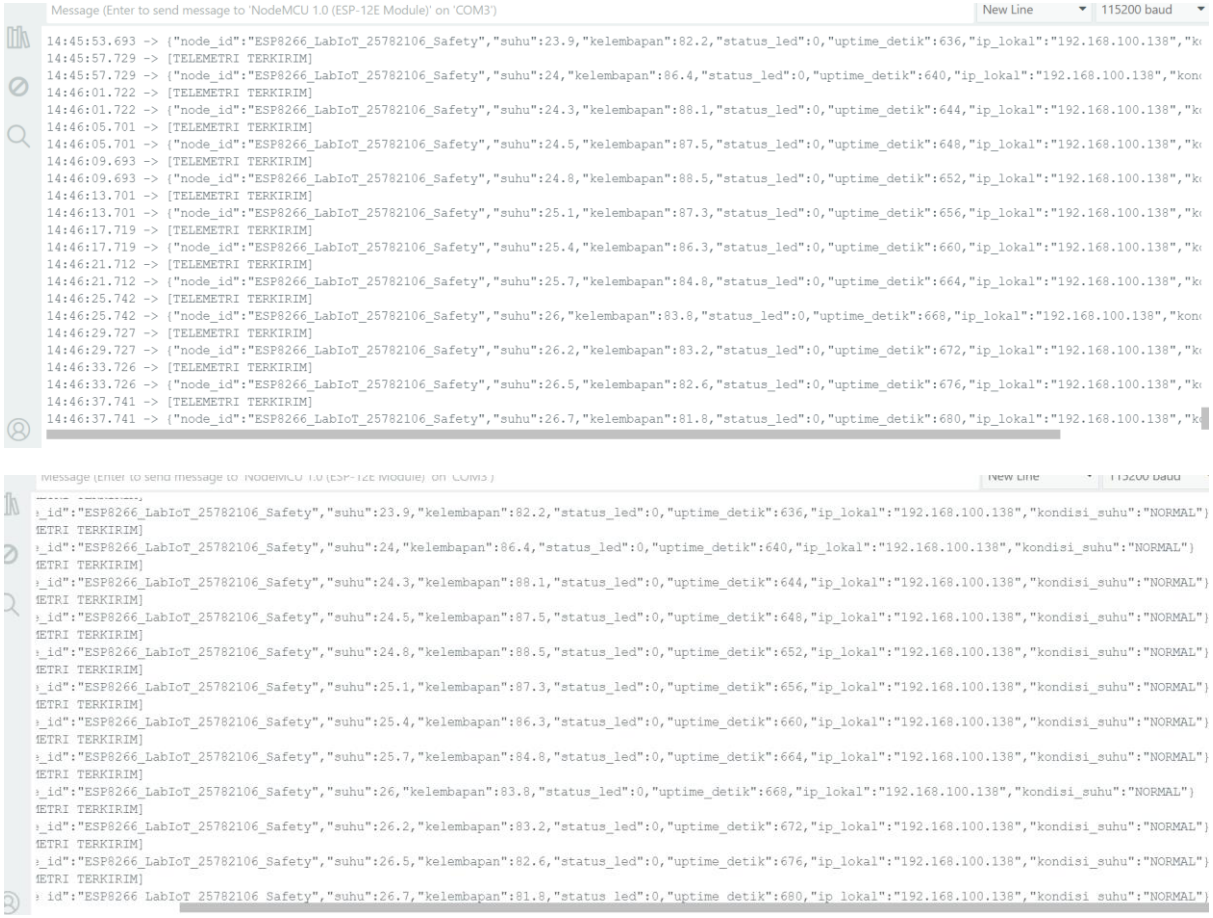
Ketika suhu sensor melewati batas 31°C, program mendeteksi kondisi panas. Sistem kemudian mengirimkan data telemetri dan peringatan melalui MQTT serta menyalakan LED sebagai indikator bahaya sesuai aturan pada program.





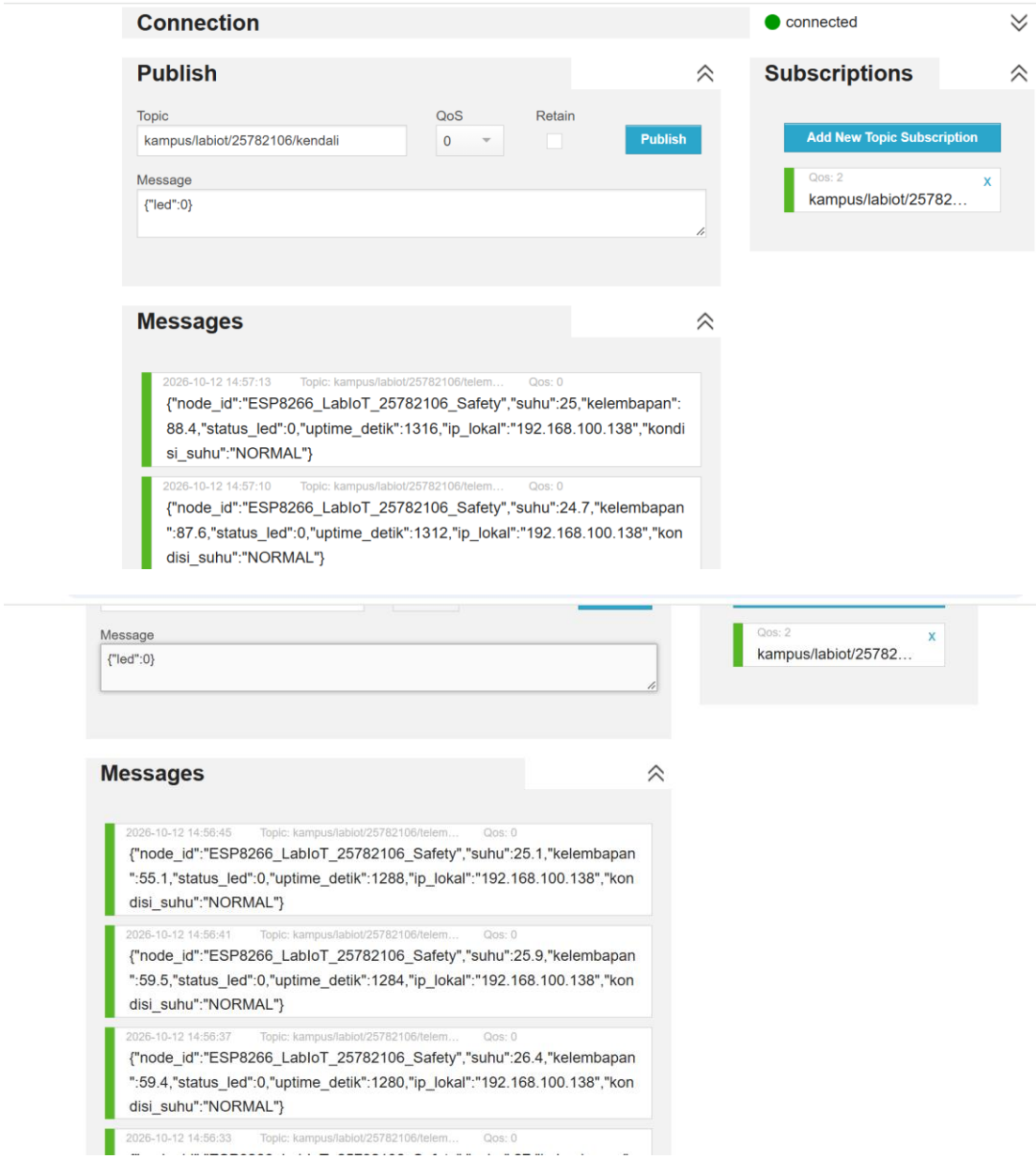
4. Hasil ketika suhu di bawah 31°C

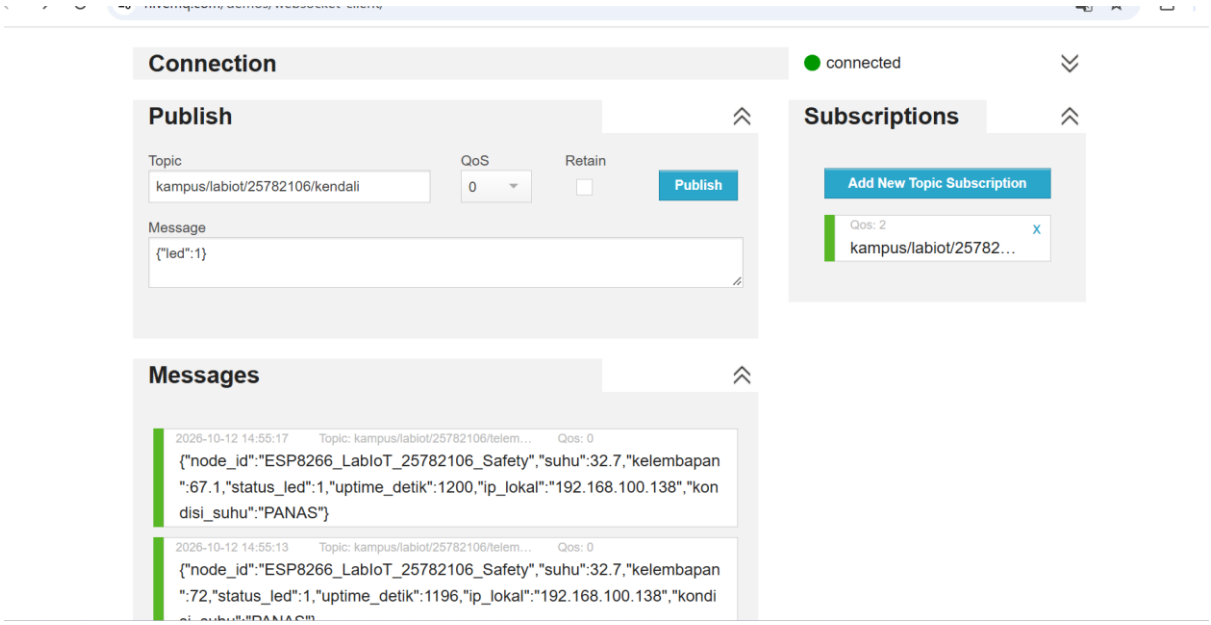
Ketika suhu berada di bawah batas 31°C, sistem kembali mendeteksi kondisi normal. Data sensor tetap dikirim melalui MQTT, sedangkan indikator LED mengikuti aturan yang sudah ditentukan dalam program.



5. Hasil kendali LED

Pengujian kendali dilakukan dengan mengirimkan perintah melalui MQTT. Payload {"led":1} digunakan untuk menyalakan LED, sedangkan {"led":0} digunakan untuk mematikannya. Hasil ini menunjukkan bahwa LED dapat dikendalikan melalui pesan MQTT.

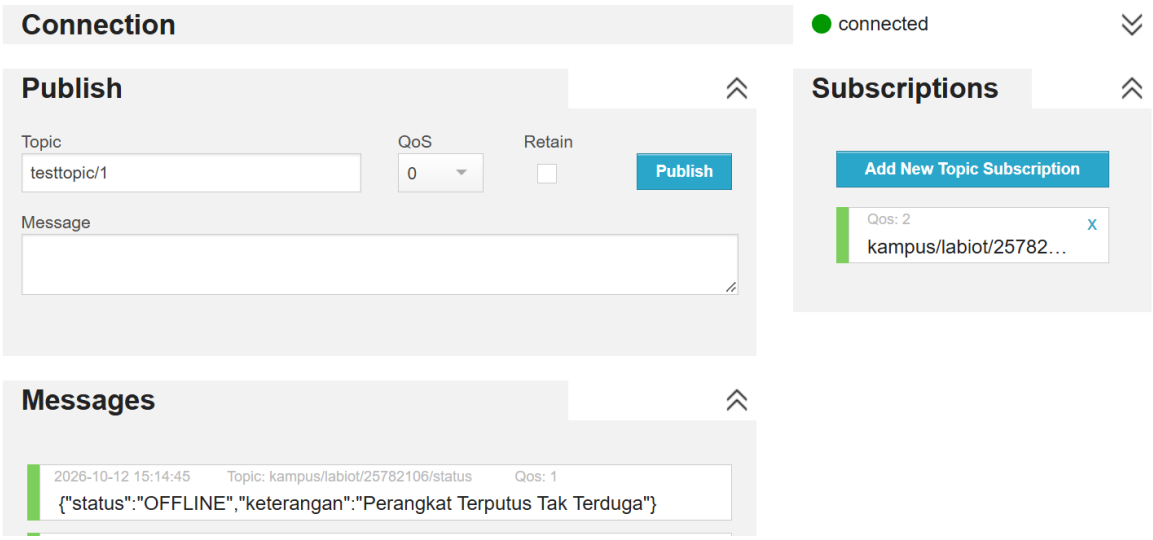

Connection
connected
Publish
Topic
kampus/labiot/25782106/kendali
QoS
0
Retain
Publish
Message
{"led":0}
Subscriptions
Add New Topic Subscription
Qos: 2
kampus/labiot/25782…
Messages
2026-10-12 14:57:13 Topic: kampus/labiot/25782106/telem… Qos: 0
{"node_id":"ESP8266_LabIoT_25782106_Safety","suhu":25,"kelembapan":88.4,"status_led":0,"uptime_detik":1316,"ip_lokal":"192.168.100.138","kondisi_suhu":"NORMAL"}
2026-10-12 14:57:10 Topic: kampus/labiot/25782106/telem… Qos: 0
{"node_id":"ESP8266_LabIoT_25782106_Safety","suhu":24.7,"kelembapan":87.6,"status_led":0,"uptime_detik":1312,"ip_lokal":"192.168.100.138","kondisi_suhu":"NORMAL"}
Message
{"led":0}
Qos: 2
kampus/labiot/25782…
Messages
2026-10-12 14:56:45 Topic: kampus/labiot/25782106/telem… Qos: 0
{"node_id":"ESP8266_LabIoT_25782106_Safety","suhu":25.1,"kelembapan":55.1,"status_led":0,"uptime_detik":1288,"ip_lokal":"192.168.100.138","kondisi_suhu":"NORMAL"}
2026-10-12 14:56:41 Topic: kampus/labiot/25782106/telem… Qos: 0
{"node_id":"ESP8266_LabIoT_25782106_Safety","suhu":25.9,"kelembapan":59.5,"status_led":0,"uptime_detik":1284,"ip_lokal":"192.168.100.138","kondisi_suhu":"NORMAL"}
2026-10-12 14:56:37 Topic: kampus/labiot/25782106/telem… Qos: 0
{"node_id":"ESP8266_LabIoT_25782106_Safety","suhu":26.4,"kelembapan":59.4,"status_led":0,"uptime_detik":1280,"ip_lokal":"192.168.100.138","kondisi_suhu":"NORMAL"}
2026-10-12 14:56:33 Topic: kampus/labiot/25782106/telem… Qos: 0



6. Hasil ketika perangkat dicabut tiba-tiba

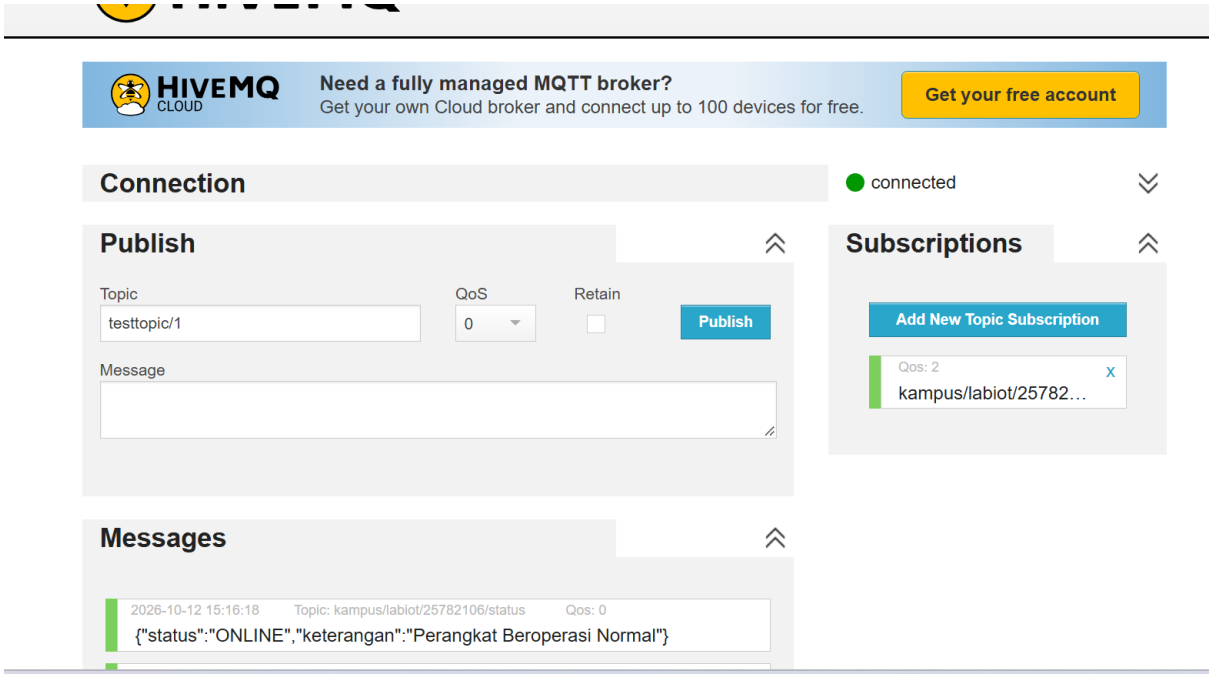
Pengujian dilakukan dengan mencabut daya NodeMCU secara tiba-tiba untuk memeriksa mekanisme Last Will and Testament (LWT). Ketika broker mendeteksi koneksi terputus secara tidak normal, pesan OFFLINE akan diterbitkan sesuai konfigurasi LWT yang telah dibuat.





7. Hasil ketika perangkat kembali online

Ketika ESP8266 kembali dinyalakan dan berhasil terhubung ke broker, perangkat dapat mengirimkan status ONLINE jika fungsi tersebut sudah dibuat pada program. Hal ini menunjukkan bahwa status koneksi perangkat dapat dipantau melalui MQTT.



Kesimpulan

Dari Praktikum Modul 6 ini, saya jadi lebih paham cara kerja MQTT untuk mengirim data sensor dan mengendalikan perangkat IoT. Saya belajar bahwa ESP8266 bisa membaca suhu dan kelembapan dari sensor DHT, lalu mengirimkan datanya ke broker MQTT untuk ditampilkan di aplikasi pemantauan.

Saya juga belajar cara mengirim perintah melalui MQTT untuk menyalakan dan mematikan LED, serta menggabungkan pemantauan sensor dengan kendali LED dalam satu program. Selain itu, saya memahami fungsi topic, payload, dan pentingnya koneksi MQTT agar data dan perintah bisa diterima dengan baik.

Pada tugas terakhir, saya belajar bahwa sistem IoT juga bisa dibuat untuk mendeteksi kondisi suhu tertentu, memberikan peringatan, dan memantau status perangkat ketika online atau offline. Secara keseluruhan, praktikum ini membuat saya lebih memahami bagaimana MQTT digunakan untuk menghubungkan sensor, mikrokontroler, broker, dan aplikasi pemantauan dalam satu sistem IoT.